MINDTOPO: Can Foundation Models Reason in Topological Space?

Anonymous Author(s)
Affiliation
Address
email

Abstract

Cognitive science treats topological reasoning, the recognition of spatial invariants such as connectivity, enclosure, and knottedness that persist under continuous deformation, as a foundational layer of human spatial cognition. We therefore ask: Can modern multimodal large language models (MLLMs) reason in topological space? We introduce **MINDTOPO**, a systematic benchmark that evaluates **topological intuition in MLLMs** across **five Piagetian primitives** (continuity, separation, order, enclosure, and knots) at two cognitive levels: **reasoning** from a static rendered scene, and **planning** through an interactive environment. Each task is procedurally generated with controllable difficulty, yielding 13 task types and 11,016 instances. We evaluate 11 MLLMs and probe two SOTA generative tools (an image model and a video model) as visual world models under a tool-use augmentation. Experiments reveal a substantial gap between frontier MLLMs and humans that widens once the model has to act: models consistently recognize a topological relation in a static scene but cannot maintain or operate on it across an action sequence. Image generation can substitute for a missing perception channel, while the video model itself violates topological constraints in the videos it produces and cannot serve as a world model that respects topology. Generative tools today can render the topology of a single scene, not simulate how that topology changes when an agent acts on it.

eifjcbvcknlugtekelgdggblrlvhdrvijrfddggbievj

Figure 1: **Overview of MINDTOPO.** We probe topological intuition across five categories (**continuity**, **separation**, **order**, **enclosure**, **knots**) at two cognitive levels: *reasoning* (visual QA over rendered scenes) and *planning* (interactive gym).

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

1 Introduction

"Representational space has to reconstruct, on its own plane and in the same order of succession, the elementary spatial relationships — first topological, then euclidean and projective."

— Jean Piaget & Bärbel Inhelder, *The Child's Conception of Space* (1956)

Spatial understanding in the physical world extends beyond Euclidean or metric properties such as distance, angle, and shape. Whether a path connects two rooms, whether a boundary fully encloses a region, whether a knot persists in a rope: these questions concern a different kind of spatial structure, one governed by **topological space** [1]. Topological properties are those spatial properties that remain invariant under continuous deformation. A rope can be bent, stretched, or coiled, but whether it is knotted does not change; a maze can be drawn in many shapes, but whether two points are connected by a path does not change. Humans exhibit strong **topological intuition** over these *invariances*: we perceive them in static scenes, we track them through *dynamic transformations* such as folding or rearranging, and we *plan* actions that create or disrupt them.

The importance of topological space in human spatial cognition has roots in cognitive science. Piaget [1] identified topological space as a distinct dimension of spatial understanding, separate from Euclidean properties, and provided a detailed classification. Chen [2, 3] demonstrated that the adult visual system extracts topological invariants before Euclidean features. Yet current evaluations of spatial reasoning in foundation models have not kept pace with this cognitive insight. Most existing works [4–6] assess Euclidean properties such as distance, direction, size, and shape. A few recent efforts [7, 8] touch on isolated topological abilities but remain limited in scope, covering only a narrow slice of what topological reasoning entails. We therefore ask: *to what extent do current foundation models possess any form of topological intuition?*

To this end, we introduce MINDTOPO (Figure 1), a benchmark for evaluating how foundation models reason in topological space. We organize topological space into five properties, informed by Piaget's classification and subsequent cognitive literature [1, 9–11]. **Continuity** asks whether two points are connected (e.g., a maze). **Separation** asks whether nearby elements are parts of one whole or distinct objects (e.g., IKEA assembly). **Order** asks about the sequential arrangement of elements along a path (e.g., tracing the color sequence of beads on a twisted string). **Enclosure** asks whether a boundary creates an inside and an outside (e.g., counting the holes in a solid object or deciding which sheep are inside a fence), with hole reasoning grounded in algebraic topology [11]. **Knots** ask whether ropes are truly knotted or loops truly interlocked.

For each property, we design tasks at two cognitive levels. **Reasoning** tasks present the model with rendered scenes and ask it to identify topological structure or infer how that structure changes under spatial transformations (e.g., whether removing a wall reconnects two regions of a maze, or whether a given rope has a knot). **Planning** tasks place the model in an interactive environment where it must execute a sequence of actions to achieve a topological goal (e.g., rotating pipe segments to establish a connected path). Each task type is backed by a parametric generation pipeline that controls difficulty through continuous parameters such as grid size, crossing number, or object count. These pipelines can produce arbitrarily large datasets. The current release contains 11,016 instances across 13 task types, with 8,016 reasoning questions and 3,000 interactive planning episodes. All scenes are rendered using Three.js or simulators, providing precise control while preserving ground-truth topological labels.

We evaluate 11 multimodal large language models on MINDTOPO and probe two SOTA generative tools (an image model and a video model) as visual world models under a tool-use augmentation. The strongest MLLM, GPT-5.5, reaches 54.13% overall, far below the human ceiling of 97.40%. Reasoning accuracy outpaces planning accuracy by a wide margin across every model tier, indicating that current MLLMs can recognize a topological relation in a single rendered scene but cannot maintain or operate on that relation across an action sequence. Within each property, performance also degrades with topological complexity rather than only on isolated hard cases, suggesting that current models lack systematic topological reasoning. The probing experiment further shows that image generation can substitute for a missing perception channel, lifting GPT-5.4-mini by more than 50 points on perception-bound tasks. The strongest open-source video generator, by contrast, violates dynamics on 116 of 119 generated rollouts, so video augmentation does not help. Generative tools, image and video alike, support a model's perception of topology but not its prediction of how topology evolves under action. Together, these findings indicate that topological intuition remains a significant blind spot across current foundation models.

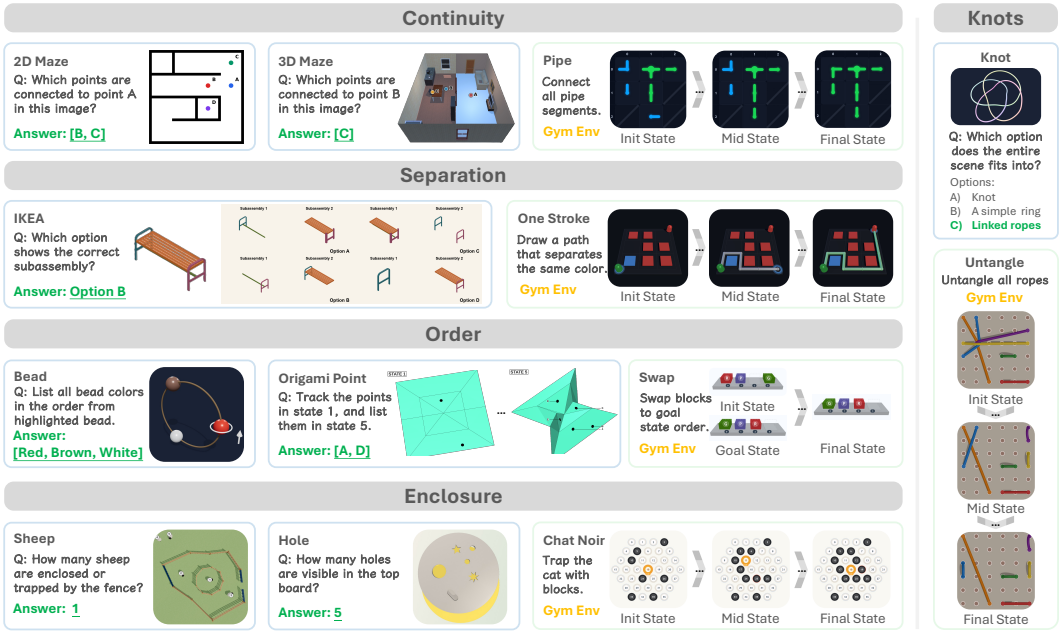


Figure 2: **MINDTOPO task overview.** 13 tasks span five topological properties (**Continuity**, **Separation**, **Order**, **Enclosure**, **Knots**) at two cognitive levels: *reasoning* (rendered scene + visual question) and *planning* (interactive environments labeled **Gym Env**).

In summary, our contributions are threefold: (1) we formalize topological intuition as a distinct and evaluable dimension of spatial understanding in foundation models, grounded in cognitive science literature on topological space; (2) we present MINDTOPO, a scalable benchmark comprising 13 task types across five topological properties and two cognitive levels, with parametric generation pipelines that enable controlled difficulty scaling; and (3) we systematically evaluate 11 multimodal large language models and probe two SOTA generative tools (image and video) as visual world models, revealing that frontier MLLMs recognize topology in a static scene but cannot operate on it under action, and that current SOTA video generators do not yet reason in topological space.

2 MINDTOPO Benchmark

2.1 Problem Formulation

Following Piaget's classification of topological space [1], we view topological intuition as the ability to perceive, infer about, and operate on spatial relations that remain invariant under continuous deformation. We probe this ability at two cognitive levels [1, 10]: *reasoning* and *planning*. At the reasoning level, each task is a visual question answering instance (o, q, a): the model is given one or more rendered images o and a question q targeting a specific topological property, and must produce a textual answer a that we verify against ground truth derived from the scene specification. At the planning level, each task is a Markov decision process $(\mathcal{S}, \mathcal{A}, T, R)$ [12], in which states are scene configurations, actions are interventions that may create or disrupt topological structure, T is the deterministic simulator transition, and R rewards reaching a configuration whose topology satisfies the goal. This unified view lets us measure, for each property, both whether a model can *recognize* the relation in a static scene and whether it can *operate on* it through a sequence of decisions.

2.2 Topological Properties

Figure 2 shows representative scenes, questions, and gym environments for every task in the suite; each property below describes its cognitive grounding, our reasoning task, and the matched planning task.

Continuity. Continuity captures whether a path or surface forms an unbroken whole, and is among the earliest spatial concepts a child acquires [1]. It underlies many practical reasoning tasks, such as navigating a maze, deciding whether a wire is severed, or threading a cable through an opening. Our reasoning task asks the model whether two marked endpoints in a rendered maze are connected. Negative cases use a single severing wall, so the answer hinges on the topological cut rather than on metric layout. We additionally include *what-if* sub-questions that probe how reachability changes if a wall is added or removed. The matched planning task places the agent in a grid of rotatable pipe segments; through 90° rotations it must form a connected source-to-destination route, probing whether it can *construct* continuity, not only recognize it.

Separation. Separation is the complement of proximity [10] and undergirds object individuation [1]. Distinguishing adjacent units as distinct is the prerequisite for any reasoning beyond an undifferentiated whole. Our reasoning task is an IKEA-style subassembly judgment: given a fully assembled object and several candidate sub-assemblies, the model selects the subset that forms a topologically separable component. We add *what-if* sub-questions asking how the partition changes if a connector is removed. The planning counterpart is a One-Stroke partitioning environment, where the agent draws a single corner-to-corner path through a colored grid that separates same-colored cells into shared regions, operationalizing the act of *creating* separation between previously contiguous regions.

Order. Order captures the sequential arrangement of elements along a path or boundary [1]. It is essential whenever a model must track *which comes before which* under a transformation of the scene. Our reasoning task adapts Piaget's bead-replication paradigm: given a curved or twisted string of colored beads, the model enumerates the sequence from a designated starting bead, or decides whether two strings preserve the same cyclic order. We further include *what-if* sub-questions that ask how the order changes under a folding or rotation of the string. The accompanying planning environment is a one-dimensional sliding-track puzzle: the agent reaches a target color permutation by sliding adjacent blocks into a single empty slot, isolating ordering reasoning from geometric and color confounds.

Enclosure. Enclosure is the inside/outside relation induced by a closed boundary, and is identified by Piaget as the topological origin of three-dimensional "insideness" [1]. The Jordan curve theorem [11] gives the same intuition mathematical form: a closed curve in the plane partitions space into an interior and an exterior, and holes in a solid correspond to homologically detectable regions of missing interior. We instantiate two reasoning tasks. *Fence & Sheep* asks the model which animals lie strictly inside a top-down enclosure boundary; *Hole Detection* asks it to count the through-holes in a solid object. Both include *what-if* sub-questions, e.g., how the count changes if a piece of material is added or removed. The matched planning task is *Chat Noir*: on a hexagonal grid the agent places one block per turn so as to encircle a moving cat before it escapes to the boundary, requiring forward reasoning about whether a partial boundary can still be closed.

Knots. Piaget originally subsumed knots under enclosure, but subsequent cognitive evidence supports treating them as a distinct ability. Strohecker characterizes knots as the "mother structure" that coordinates all other topological relations [9]. Croom and Firestone show that humans reason about knots far worse than they perceive them, dissociating knot understanding from domain-general physical reasoning [13]. Our reasoning task presents one or more rendered closed rope loops and asks a battery of topology questions: whether a single loop is truly knotted or just visually tangled, whether two loops are linked, and *what-if* variants asking how the answer changes if a crossing is flipped. The matched planning environment is *Untangle*: given a board of plug positions joined by ropes, the agent moves plugs across a discrete grid until no two ropes cross.

2.3 Data Collection and Statistics

Figure 3 shows the four-stage pipeline that builds every task in MINDTOPO. *Scene Generation* procedurally synthesizes scenes for all thirteen task types from their native simulators or rendering scripts. *Topological Annotation* attaches the ground-truth label its property requires, including which points are connected versus isolated in 2D Maze, whether same-color regions are separated in One Stroke, or whether a closed loop is truly knotted in Knots. *Difficulty Control* varies scene parameters that change topological complexity, such as wall count, grid size, or number of crossings, and produces easy, medium, and hard variants. *Instances Generation* turns each annotated scene into a reasoning instance with a templated question and answer, and into a matched planning instance whose

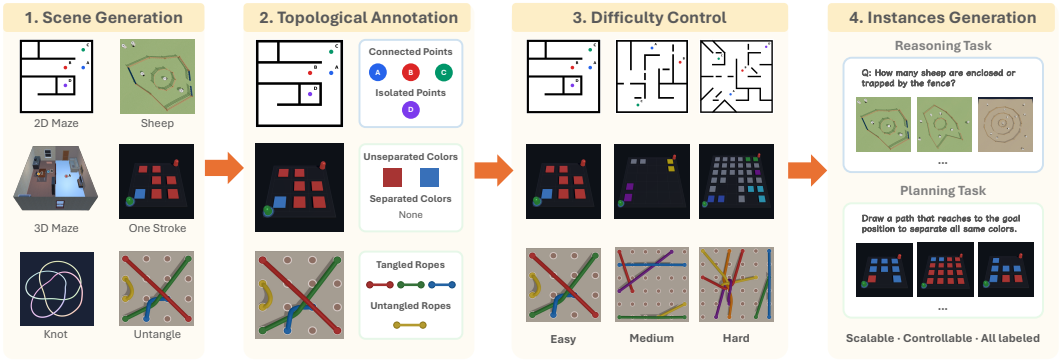


Figure 3: Data Collection Pipeline for MINDTOPO.

initial state, goal, and reward come from the same annotation. The pipeline is scalable, supports controllable difficulty, and labels every instance by construction.

Figure 4 summarizes the resulting dataset. MINDTOPO contains 11,016 instances across the thirteen task types and the five topological properties, split 73% reasoning and 27% planning. The eight reasoning tasks (2D Maze, 3D Maze, IKEA, Bead, Origami Point, Sheep, Hole, Knots) each contribute roughly 1,000 instances, and the five planning tasks (Pipe, One Stroke, Swap, Chat noir, Untangle) each contribute 600.

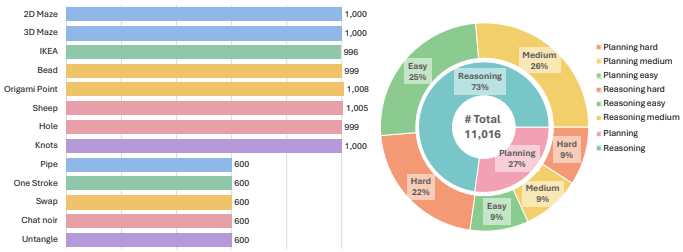


Figure 4: Data statistics for MINDTOPO.

3 Experiments

3.1 Experimental Setups

Models. We benchmark 11 MLLMs, four proprietary and seven open-weight. The proprietary tier includes Gemini 3.1 Flash-Lite [14], Gemini 3.1 Pro [15], GPT-5.4 mini [16], and GPT-5.5 [17]. The open-weight tier includes InternVL3.5-241B-A28B [18], Nemotron Nano 12B v2 VL [19], Gemma-4-31B-IT [20], Qwen3.5-397B-A17B [21], Cosmos-Reason2 [22], BAGEL-7B [23], and ThinkMorph-7B [24]. We additionally report human performance and random-chance baselines.

Human evaluation. Trained annotators answer a subset of the benchmark under the same instructions as the models. Three independent labels per item are collected, with raw percent agreement before adjudication reaching 0.90. Full details appear in Appendix B.1.1 and Appendix B.1.2.

Protocol. All models are evaluated with deterministic decoding (temperature 0). Reasoning tasks are scored by exact match for scalar and multiple-choice answers and by set equality for unordered list answers. Planning tasks are scored by executing the predicted actions in the corresponding environment and counting an episode as successful only when it reaches the task-defined terminal condition. Primary metrics are task accuracy for reasoning and episode success rate for planning.

3.2 Benchmark Results

Models recognize topology but fail to operate on it. Table 1 reports accuracy across the 13 task types. The leading model, GPT-5.5, reaches 54.13% overall, far below the human ceiling of 97.40%. The largest gap is between reasoning and planning. Gemini 3.1 Pro drops from 60.00% on reasoning to 43.20% on planning, GPT-5.5 from 57.74% to 48.33%, and Gemma-4-31B-IT from 39.74% to 2.50%. Current MLLMs identify a topological relation in a single rendered scene, yet topology breaks once they have to act on it.

Model	Continuity			Separation		Order			Enclosure			Knots	
	2D Maze	3D Maze	Pipe	IKEA	One Stroke	Bead	Origami Point	Swap	Sheep	Hole	Chat noir	Knots	Untangle
Proprietary Models													
Gemini 3.1 Flash Lite	21.90	60.30	0.00	42.47	0.00	66.97	36.70	12.80	61.00	55.00	25.60	62.40	26.90
Gemini 3.1 Pro	44.10	75.60	26.56	39.80	29.43	75.90	32.00	71.70	78.60	63.00	58.90	71.00	29.40
GPT-5.4 mini	20.40	30.10	0.00	36.24	0.00	59.86	21.73	15.60	29.25	57.98	26.10	26.00	7.80
GPT-5.5	50.90	58.60	21.67	42.67	25.00	72.37	31.75	100.00	78.21	69.26	61.70	58.20	33.30
Open-Weight Models													
InternVL-3.5-241b-a28b	10.80	34.50	1.20	17.80	0.00	57.30	25.40	12.80	28.30	50.00	6.70	37.80	0.60
Nemotron Nano 12B v2 VL	18.30	25.20	0.50	41.00	0.60	41.50	24.50	13.90	18.00	59.70	1.70	24.80	15.00
Gemma-4-31b-it	26.50	55.80	7.50	18.80	0.00	66.37	45.24	0.00	50.35	49.90	0.00	5.00	6.06
Qwen3.5-VL-397B	18.72	67.77	0.00	39.76	0.00	62.19	23.21	59.50	42.99	54.49	29.70	45.60	10.30
Cosmos-Reason2	30.10	22.40	0.00	35.74	2.20	56.96	9.82	19.40	29.05	41.82	12.20	20.00	21.70
BAGEL-7B	7.30	33.30	0.00	21.18	0.00	17.42	13.69	0.00	34.73	57.98	0.00	36.80	7.20
ThinkMorph-7B	8.20	31.10	0.00	21.18	0.00	17.22	11.81	0.00	20.00	55.89	0.00	27.40	0.00
Random Chance	11.12	22.83	0.00	19.94	0.00	8.43	10.07	8.89	10.31	8.41	1.67	15.01	6.67
Human	**97.18**	**98.62**	**100.00**	**94.37**	**100.00**	**95.77**	**91.94**	**100.00**	**99.06**	**97.65**	**100.00**	**91.55**	**100.00**

Table 1: **Evaluation on MINDTOPO.** Reasoning task denotes non-interactive perception tasks, and Planning task denotes interactive/planning tasks.

The action gap widens with model tier. The two frontier proprietary models retain a substantial fraction of their reasoning competence in the planning regime, while the open-weight tier collapses. The best open-weight model on planning, Qwen3.5-VL-397B, reaches 19.90% on average, and most open-weight models score 0% on Pipe and One Stroke. The gap between reasoning and planning is therefore not a uniform property of the benchmark. It widens monotonically as we move down the model tier, suggesting that the planning bottleneck does not close with current open-source scaling.

No single model dominates across topological primitives. Per-property leadership splits between Gemini 3.1 Pro (continuity 48.7%, separation 34.6%, knots 50.2%) and GPT-5.5 (order 68.0%, enclosure 69.7%). Within a single property the reasoning leader and the planning leader often differ. Gemini wins the static Knot identification task (71.0%) but trails on Untangle (29.4% vs. GPT-5.5's 33.3%), and the same flip appears between Hole reasoning and Chat-Noir planning. The five Piagetian primitives engage different MLLM weaknesses, so a model's strength on one primitive is not predictive of its strength on the others.

Three planning environments resist every model evaluated. Pipe, One Stroke, and Untangle stay below 35% across all 11 MLLMs, with best scores of 26.6% (Gemini, Pipe), 29.4% (Gemini, One Stroke), and 33.3% (GPT-5.5, Untangle). Most open-weight models score near 0% on the three. These environments share a structural feature. Success requires preserving a topological invariant (continuity, separation, or knot class) across many constrained actions, with no metric shortcut available. Frontier reasoning capacity does not yet translate into success when topology must be maintained over a trajectory.

One planning environment is already saturable. GPT-5.5 solves every Swap instance, and Qwen3.5-VL-397B reaches 59.50%, the only open-weight non-trivial planning result on the benchmark. Swap reduces topological state to a discrete permutation, which frontier MLLMs can plan over symbolically. Pipe, One Stroke, and Untangle do not admit such a reduction. The contrast marks where current models stop being able to plan. When the topological state cannot be read off finite discrete tokens, the action search collapses. Full per-task and per-property breakdowns appear in Appendix B.

3.3 Error Analysis

Methodology. We hand-label all 855 wrong predictions produced by Gemini-3.1-Pro and InternVL3.5-241B, our representatives of the proprietary and open-weight tiers. Each prediction is assigned a single primary category from a seven-class taxonomy that runs along the model's processing pipeline, from surface format failures to deep planning errors. Categories are assigned in fixed priority order so an early failure is never credited to a later stage. Figure 5 shows one representative example per category. Full taxonomy definitions and per-property frequencies appear in Appendix C.1.

Failure modes invert between reasoning and planning. Figure 6 shows the two regimes fail in qualitatively different ways. Reasoning errors concentrate at the early pipeline. Perception grounding

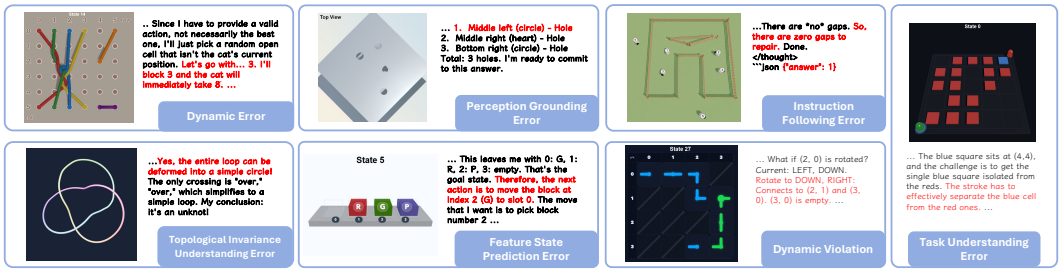


Figure 5: Qualitative examples of each error category. The red span marks the offending step in the model's reasoning.

and topological invariance together account for 84.3% of Gemini-3.1-Pro's errors and 76.5% of InternVL3.5-241B's. Planning errors concentrate late. Gemini fails at the planner (54.0% action planning, 32.1% dynamic violation), and InternVL fails earlier still, at task understanding (58.8%, mostly multi-turn drift). Reasoning is gated at perception. Planning is gated at action selection.

Reasoning failures are dominated by perception and false invariance. Within reasoning errors, perception grounding alone covers 48.7% of Gemini-3.1-Pro's mistakes and 54.7% of InternVL3.5-241B's, dominated by missed or hallucinated visual elements (an overlooked hole in a fenced region, an unregistered wall in a maze, a printed shading pattern read as a real opening). Object misidentification follows, especially the blue-versus-red rope confusion in knot tasks and the open-versus-closed misread in pipe tasks. Topological invariance contributes a further 35.6% (Gemini-3.1-Pro) and 21.8% (InternVL3.5-241B). The dominant subtype is *false invariance*, where the model treats two configurations as equivalent because endpoint coordinates look unchanged even after a crossing or enclosure has flipped. Format errors expose the sharpest tier gap (6.6% vs. 20.3%), almost entirely from the open-weight model committing one value in chain-of-thought and a different value in the structured answer field.

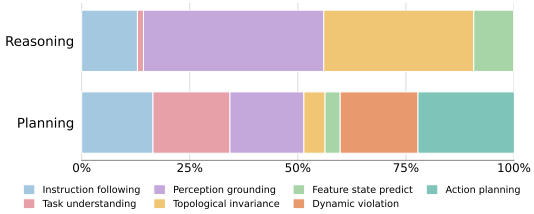


Figure 6: Error category distribution on reasoning vs. planning tasks from Gemini-3.1-Pro and InternVL3.5-241B.

Planning failures split between action choice and dynamic legality. For Gemini-3.1-Pro, action planning (54.0%) and dynamic violations (32.1%) account for 86.1% of planning errors. The most common action-planning subtype is short-horizon traps, where a locally reasonable step creates a worse downstream state. Chat-Noir plays block one escape route while opening another, one-stroke moves cut off an unvisited region, and Swap moves increase the minimum remaining distance to the goal. Dynamic violations are dominated by topological-barrier crossings rather than physical collisions, with the model proposing to untangle a knot by passing one strand through another. InternVL3.5-241B fails earlier than this. 58.8% of its planning errors come from task understanding, almost all multi-turn drift, where the model identifies the task on the first turn and reframes it on later turns (pipe-routing puzzles reinterpreted as Unblock Me, Chat-Noir games hypothesized to encode prime-number patterns). Only 14.4% of InternVL's planning errors come from the planner itself, so the open-weight tier loses the thread of the task before bad action selection becomes the bottleneck. Subtype distributions are deferred to Appendix C.3.

3.4 Probing Experiment

Methodology. Section 3.3 shows that planning failures on MINDTOPO are largely dynamic rather than declarative. Frontier models read the scene correctly but lose topological state across actions. We probe whether explicit visual prediction can supply that state. All three configurations use GPT-5.4-mini as the planner on four tasks (Sheep, 2D Maze, Untangle, One Stroke). The **baseline** acts directly from the rendered observation. The **interleaved** variant calls GPT-Image-2 [25] at every step to render the predicted next state. The **video** variant calls Wan2.2-I2V-A14B [26] once per episode to roll out a candidate plan.

Difficulty	Sheep			2D Maze			Untangle			One Stroke		
	E	M	H	E	M	H	E	M	H	E	M	H
Baseline	36.7	26.9	24.2	20.7	23.4	17.2	23.3	0.0	0.0	0.0	0.0	0.0
Video	–	–	–	–	–	–	50.0	10.0	0.0	0.0	0.0	5.0
Interleaved	100.0	70.0	70.0	70.0	75.0	60.0	55.0	20.0	10.0	0.0	0.0	0.0

Table 2: Probing success rate (%) by difficulty (E = Easy, M = Medium, H = Hard). All variants use GPT-5.4-mini.

Image augmentation rescues local topology but not global invariants. Interleaved image generation lifts Sheep from 29.3% to 80.0% and 2D Maze from 20.4% to 68.3%, where the relevant relation is visible in a single rendered frame (Table 2). It roughly doubles Untangle on average (7.8% to 28.3%) but cannot recover hard knots, where a misplaced over- versus under-crossing flips the topological class. On One Stroke the planner stays at chance across all variants, because the Eulerian property of the entire trace cannot be read from any single image. Image generation supplies what the planner needs to perceive the scene. It does not supply a topological representation.

SOTA video generators do not reason in topological space, and video augmentation does not help. Video rollouts from Wan2.2-I2V-A14B leave GPT-5.4-mini essentially neutral on Untangle and One Stroke. Figure 7 explains why. The strongest open-source video generator violates dynamics in 116 of 119 generated rollouts, with topological-invariance errors on 78% and consistency errors on 76%. The tool fails the same topology test the planner is being asked to pass, so it cannot stand in for a topology-aware world model. Current generative tools, image and video alike, can support a model's perception of topology but not its prediction of how topology evolves under action.

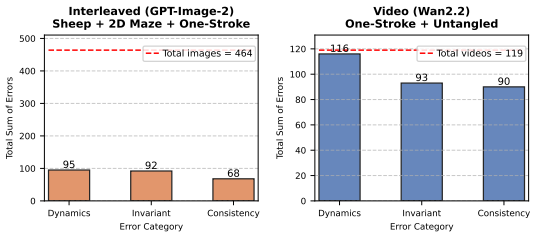


Figure 7: Per-generation errors by category. **Left**: 464 GPT-Image-2 images (Sheep, 2D Maze, One Stroke). **Right**: 119 Wan2.2 videos (One Stroke, Untangle). Wan2.2 violates dynamics on 116 of 119 videos.

What the two probes tell us together. Image and video augmentation localize the planning bottleneck along the same axis. An image generator helps because it converts a perception-bound topological task into one the planner already handles step by step. A video generator does not help because it inherits, rather than relieves, the very invariance failures the planner exhibits on action sequences. The asymmetry implies that topological intuition behaves differently from generic visual intuition: it can be augmented at the perception step, where a single frame suffices, but not at the prediction step, where a sequence of actions must respect the same invariant. Closing the planning gap on MINDTOPO therefore requires that topological state be carried inside the model that acts, or inside a generative world model whose rollouts respect topological invariants by construction.

4 Related Work

Topological Cognition. Cognitive development places topology before projective and Euclidean reasoning as a primitive layer of spatial cognition [1, 27, 28]. Adult vision shows the same priority. The visual system reads off topological invariants such as connectedness, hole count, and knot structure before feature-based attributes [2, 3, 29–31, 9, 13]. Formal topology [11, 10] and conceptual-space theory [32] provide the five-class taxonomy we adopt.

Cognitively Grounded Benchmarks. Several recent benchmarks probe foundation models through a developmental or topology-adjacent lens. Piaget-inspired diagnostics target conservation, perspective-taking, reversed cognitive development, visual analogy, and pre-linguistic vision [33–38]. Other work isolates a single topology-adjacent task such as path connectivity, knots, folding, abstract positional reasoning, or constrained-manifold puzzles [39, 8, 40–43]. Closest in spirit, recent work [44] contrasts topological and geometric concept sensitivity in vision models. As Table 3 shows, KnotGym is the

only prior topological entry, and it covers a single property (knots) on 20 instances. MINDTOPO grounds all five Piagetian primitives in a single evaluation suite at scale.

Spatial Reasoning Benchmarks for MLLMs. Spatial benchmarks for MLLMs largely target Euclidean and viewpoint-centric competence rather than topological invariance. Static VQA suites cover metric properties such as distance, direction, and inter-object relations [4, 45, 7, 46–49]. Cognitive-map and viewpoint-integration benchmarks evaluate spatial memory and limited-view inference [5, 6, 50], and interactive evaluations test multi-step spatial control and affordance use [51–53]. Table 3 makes the contrast precise. Every prior Euclidean entry evaluates via static question answering with no interactive component, and no entry combines cognitive grounding, controllable difficulty, and a scalable generation pipeline. MINDTOPO provides all four properties under a topological taxonomy of five primitives, with reasoning questions and interactive planning sharing the same task suite. A longer per-work discussion, including a section on generative world models and physical-reasoning benchmarks, appears in Appendix D.

Benchmark	Spatial Type	#Topo. Properties	#Tasks	#Instances	QA	Interactive Env.	Cognitive Grounding	Difficulty Control	Scalability
SpatialVLM [4]	Euclidean	N/A	2	546	✓	✗	✗	✗	✓
SpatialMQA [45]	Euclidean	N/A	6	5,392	✓	✗	✗	✗	✗
OmniSpatial† [7]	Euclidean†	N/A	50	8,400	✓	✗	✓	✗	✗
SITE [46]	Euclidean	N/A	6	8,068	✓	✗	✓	✗	✗
3DSRBench [47]	Euclidean	N/A	12	2,772	✓	✗	✗	✗	✗
Mind the Gap [49]	Euclidean	N/A	6	1,800	✓	✗	✓	✓	✓
VSI-Bench [5]	Euclidean	N/A	8	5,130	✓	✗	✗	✗	✗
MindCube [6]	Euclidean	N/A	3	21,154	✓	✗	✓	✗	✗
RoboSpatial [53]	Euclidean	N/A	3	350	✓	✗	✗	✗	✓
KnotGym [8]	Topological	1 (Knot)	3	9	✗	✓	✗	✓	✓
MINDTOPO (ours)	Topological	5	13	11,016	✓	✓	✓	✓	✓

Table 3: Comparison with related benchmarks. MINDTOPO is the first benchmark to systematically evaluate topological spatial reasoning, grounded in cognitive science. † Touches on topological properties but does not explicitly evaluate them as a topological focus.

5 Conclusion and Limitation

Conclusion. We presented MINDTOPO, a systematic benchmark for evaluating topological intuition in multimodal large language models. Grounded in Piaget's classification, the benchmark covers five topological properties (continuity, separation, order, enclosure, and knots) at two cognitive levels (reasoning and planning) across 13 procedurally generated task types. Across 11 MLLMs spanning four proprietary and seven open-weight tiers, the central finding is that topological intuition remains a blind spot. Models recognize a topological relation in a static rendered scene but cannot maintain or operate on it across an action sequence. A tool-use probe with two SOTA generative models shows that image generation can substitute for missing perception, but the strongest open-source video generator violates dynamics on nearly every rollout it produces, so video augmentation does not help. Generative tools, image and video alike, support a model's perception of topology but not its prediction of how topology evolves under action. We hope MINDTOPO and its parametric pipeline serve as a controlled diagnostic for future foundation models that aim to internalize the qualitative structure of the physical world.

Limitation. MINDTOPO has limitations that point to natural directions for future work. First, all scenes in the benchmark are procedurally rendered via Three.js or task-specific simulators, which gives clean ground truth but lacks the visual variability of real-world photographs. Second, the probing experiment evaluates only one SOTA image model (GPT-Image-2) and one SOTA open-source video model (Wan2.2-I2V-A14B). A broader sweep across more recent generative tools would strengthen the claim that current generative models cannot serve as topology-aware world models. Third, the five Piagetian primitives covered here do not exhaust topological space. Topology-flavored relations such as orientation, continuous deformation under composed transformations, and higher-genus surfaces remain out of scope and could be added in future releases. Fourth, the probing experiment uses 60 samples per environment. This is sufficient for the coarse comparisons we draw, but denser sampling would tighten statistical bounds on subtle effects.

References

[1] Jean Piaget. *Child's Conception of Space: Selected Works vol 4*. Routledge, 2013. 2, 3, 4, 8, 21, 22, 28, 39

[2] Lin Chen. Topological structure in visual perception. *Science*, 218(4573):699–700, 1982. 2, 8, 40

[3] Lin Chen. The topological approach to perceptual organization. *Visual Cognition*, 12(4):553–637, 2005. 2, 8, 40

[4] Boyuan Chen, Zhuo Xu, Sean Kirmani, Brain Ichter, Dorsa Sadigh, Leonidas Guibas, and Fei Xia. Spatialvlm: Endowing vision-language models with spatial reasoning capabilities. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14455–14465, 2024. 2, 9, 41

[5] Jihan Yang, Shusheng Yang, Anjali W Gupta, Rilyn Han, Li Fei-Fei, and Saining Xie. Thinking in space: How multimodal large language models see, remember, and recall spaces. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 10632–10643, 2025. 9, 41

[6] Qineng Wang, Baiqiao Yin, Pingyue Zhang, Jianshu Zhang, Kangrui Wang, Zihan Wang, Jieyu Zhang, Keshigeyan Chandrasegaran, Han Liu, Ranjay Krishna, et al. Mindcube: Spatial mental modeling from limited views. *arXiv e-prints*, pages arXiv–2506, 2025. 2, 9, 41

[7] Mengdi Jia, Zekun Qi, Shaochen Zhang, Wenyao Zhang, Xinqiang Yu, Jiawei He, He Wang, and Li Yi. Omnispatial: Towards comprehensive spatial reasoning benchmark for vision language models. *arXiv preprint arXiv:2506.03135*, 2025. 2, 9, 41

[8] Zizhao Chen and Yoav Artzi. Knot so simple: A minimalistic environment for spatial reasoning. *arXiv preprint arXiv:2505.18028*, 2025. 2, 8, 9, 40

[9] Carol Strohecker. *Why knot?* PhD thesis, Massachusetts Institute of Technology, 1991. 2, 4, 8, 21, 40

[10] J Larry Martin. An analysis of some of piaget's topological tasks from a mathematical point of view. *Journal for research in mathematics education*, 7(1):8–24, 1976. 3, 4, 8, 21, 22, 40

[11] Allen Hatcher. *Algebraic Topology*. Cambridge University Press, 2002. 2, 4, 8, 21, 22, 30, 40

[12] Richard S Sutton, Andrew G Barto, et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998. 3

[13] Sholei Croom and Chaz Firestone. Tangled physics: knots strain intuitive physical reasoning. *Open Mind*, 8:1170–1190, 2024. 4, 8, 22, 40

[14] The Gemini Team. Gemini 3.1 Flash-Lite: Built for intelligence at scale. https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-flash-lite, 2026. 5

[15] The Gemini Team. Gemini 3.1 Pro: A smarter model for your most complex tasks. https://blog.google/innovation-and-ai/models-and-research/gemini-models/gemini-3-1-pro/, 2026. 5

[16] OpenAI. Introducing GPT-5.4 mini and nano. https://openai.com/index/introducing-gpt-5-4-mini-and-nano/, 2026. 5

[17] OpenAI. GPT-5.5 system card. https://openai.com/index/gpt-5-5-system-card/, 2026. 5

[18] Weiyun Wang, Zhangwei Gao, Lixin Gu, Hengjun Pu, Long Cui, Xingguang Wei, Zhaoyang Liu, Linglin Jing, Shenglong Ye, Jie Shao, et al. InternVL3.5: Advancing open-source multimodal models in versatility, reasoning, and efficiency. *arXiv preprint arXiv:2508.18265*, 2025. 5

[19] NVIDIA, Amala Sanjay Deshmukh, Kateryna Chumachenko, Tuomas Rintamaki, Matthieu Le, Tyler Poon, Danial Mohseni Taheri, Ilia Karmanov, Guilin Liu, Jarno Seppanen, Guo Chen, et al. NVIDIA Nemotron Nano V2 VL. *arXiv preprint arXiv:2511.03929*, 2025. 5

[20] Google DeepMind. Gemma 4 model card. https://deepmind.google/models/gemma/gemma-4/, 2026. 5

[21] Qwen Team. Qwen3.5: Towards native multimodal agents. https://qwen.ai/blog?id=qwen3.5, February 2026. 5

[22] NVIDIA. Cosmos-Reason2. https://docs.nvidia.com/cosmos/latest/reason2/index.html, 2026. 5

[23] Chaorui Deng, Deyao Zhu, Kunchang Li, Chenhui Gou, Feng Li, Zeyu Wang, Shu Zhong, Weihao Yu, Xiaonan Nie, Ziang Song, Guang Shi, and Haoqi Fan. Emerging properties in unified multimodal pretraining. *arXiv preprint arXiv:2505.14683*, 2025. 5

[24] Jiawei Gu, Yunzhuo Hao, Huichen Will Wang, Linjie Li, Michael Qizhe Shieh, Yejin Choi, Ranjay Krishna, and Yu Cheng. ThinkMorph: Emergent properties in multimodal interleaved chain-of-thought reasoning. *arXiv preprint arXiv:2510.27492*, 2025. 5

[25] OpenAI. Introducing ChatGPT Images 2.0. https://openai.com/index/introducing-chatgpt-images-2-0/, April 2026. Accessed: 2026-05-07. 7

[26] Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwu Yu, Haiming Zhao, Jianxiao Yang, Jianyuan Zeng, Jiayu Wang, Jingfeng Zhang, Jingren Zhou, Jinkai Wang, Jixuan Chen, Kai Zhu, Kang Zhao, Keyu Yan, Lianghua Huang, Mengyang Feng, Ningyi Zhang, Pandeng Li, Pingyu Wu, Ruihang Chu, Ruili Feng, Shiwei Zhang, Siyang Sun, Tao Fang, Tianxing Wang, Tianyi Gui, Tingyu Weng, Tong Shen, Wei Lin, Wei Wang, Wei Wang, Wenmeng Zhou, Wente Wang, Wenting Shen, Wenyuan Yu, Xianzhong Shi, Xiaoming Huang, Xin Xu, Yan Kou, Yangyu Lv, Yifei Li, Yijing Liu, Yiming Wang, Yingya Zhang, Yitong Huang, Yong Li, You Wu, Yu Liu, Yulin Pan, Yun Zheng, Yuntao Hong, Yupeng Shi, Yutong Feng, Zeyinzi Jiang, Zhen Han, Zhi-Fan Wu, and Ziyu Liu. Wan: Open and advanced large-scale video generative models, 2025. 7

[27] Jean Piaget, Wolfe Mays, and Evert Willem Beth. *Mathematical epistemology and psychology*. D. Reidel, 1966. 8, 40

[28] Seymour Papert. Mindstorms: children, computers, and powerful ideas, 1980. 8, 40

[29] Roberto Casati and Achille C Varzi. *Holes and other superficialities*. The MIT Press, 1994. 8, 40

[30] Rolf Nelson and Stephen E Palmer. Of holes and wholes: The perception of surrounded regions. *Perception*, 30(10):1213–1226, 2001. 40

[31] Marco Bertamini and Camilla J Croucher. The shape of holes. *Cognition*, 87(1):33–54, 2003. 8, 40

[32] P Gärdenfors. Conceptual spaces-the geometry of thought. 2000. 8, 40

[33] Yijiang Li, Qingying Gao, Haoran Sun, Haiyun Lyu, Dezhi Luo, and Hokin Deng. Cogdevelop2k: Reversed cognitive development in multi-modal large language models. 2024. 8, 40

[34] Xinglin Wang, Peiwen Yuan, Shaoxiong Feng, Yiwei Li, Boyuan Pan, Heda Wang, Yao Hu, and Kan Li. Coglm: Tracking cognitive development of large language models. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 73–87, 2025. 40

[35] Dezhi Luo, Haiyun Lyu, Qingying Gao, Haoran Sun, Yijiang Li, and Hokin Deng. Vision language models know law of conservation without understanding more-or-less. *arXiv preprint arXiv:2410.00332*, 2024. 40

[36] Qingying Gao, Yijiang Li, Haiyun Lyu, Haoran Sun, Dezhi Luo, and Hokin Deng. Vision language models see what you want but not what you see. *arXiv preprint arXiv:2410.00324*, 2024. 40

[37] Eunice Yiu, Maan Qraitem, Anisa Noor Majhi, Charlie Wong, Yutong Bai, Shiry Ginosar, Alison Gopnik, and Kate Saenko. Kiva: Kid-inspired visual analogies for testing large multimodal models. *arXiv preprint arXiv:2407.17773*, 2024. 40

[38] Liang Chen, Weichu Xie, Yiyan Liang, Hongfeng He, Hans Zhao, Zhibo Yang, Zhiqi Huang, Haoning Wu, Haoyu Lu, Yiping Bao, et al. Babyvision: Visual reasoning beyond language. *arXiv preprint arXiv:2601.06521*, 2026. 8, 40

[39] Alan Dao and Dinh Bach Vu. Alphamaze: Enhancing large language models' spatial intelligence via grpo. *arXiv preprint arXiv:2502.14669*, 2025. 8, 40

[40] Rui Xu, Dakuan Lu, Zicheng Zhao, Xiaoyu Tan, Xintao Wang, Siyu Yuan, Jiangjie Chen, and Yinghui Xu. Origamispace: Benchmarking multimodal llms in multi-step spatial reasoning with mathematical constraints. *arXiv preprint arXiv:2511.18450*, 2025. 8, 40

[41] Ryan Spencer, Roey Yaari, Ritvik Vemavarapu, Joyce Yang, Steven Ngo, and Utkarsh Sharma. Gamibench: Evaluating spatial reasoning and 2d-to-3d planning capabilities of mllms with origami folding tasks. *arXiv preprint arXiv:2512.22207*, 2025. 40

[42] Christopher Driggers-Ellis, Gabriel Ayoubi, and Christan Grant. Optical: An abstract positional reasoning benchmark for vision language models. In *2025 IEEE International Conference on Data Mining Workshops (ICDMW)*, pages 1437–1441. IEEE, 2025. 40

[43] Chen Yang, Guanxin Lin, Youquan He, Peiyao Chen, Guanghe Liu, Yufan Mo, Zhouyuan Xu, Linhao Wang, Guohui Zhang, Zihang Zhang, et al. Thinking in structures: Evaluating spatial intelligence through reasoning on constrained manifolds. *arXiv preprint arXiv:2602.07864*, 2026. 8, 40

[44] Zekun Wang and Sashank Varma. Computer vision models show human-like sensitivity to geometric and topological concepts. *arXiv preprint arXiv:2505.13281*, 2025. 8, 40

[45] Jingping Liu, Ziyan Liu, Zhedong Cen, Yan Zhou, Yinan Zou, Weiyan Zhang, Haiyun Jiang, and Tong Ruan. Can multimodal large language models understand spatial relations? In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 620–632, 2025. 9, 41

[46] Wenqi Wang, Reuben Tan, Pengyue Zhu, Jianwei Yang, Zhengyuan Yang, Lijuan Wang, Andrey Kolobov, Jianfeng Gao, and Boqing Gong. Site: towards spatial intelligence thorough evaluation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 9058–9069, 2025. 9, 41

[47] Wufei Ma, Haoyu Chen, Guofeng Zhang, Yu-Cheng Chou, Jieneng Chen, Celso de Melo, and Alan Yuille. 3dsrbench: A comprehensive 3d spatial reasoning benchmark. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 6924–6934, 2025. 9, 41

[48] Xinmiao Huang, Qisong He, Zhenglin Huang, Boxuan Wang, Zhuoyun Li, Guangliang Cheng, Yi Dong, and Xiaowei Huang. Spatial-dise: A unified benchmark for evaluating spatial reasoning in vision-language models. *arXiv preprint arXiv:2510.13394*, 2025. 41

[49] Ilias Stogiannidis, Steven McDonagh, and Sotirios A Tsaftaris. Mind the gap: Benchmarking spatial reasoning in vision-language models. *arXiv preprint arXiv:2503.19707*, 2025. 9, 41

[50] Pingyue Zhang, Zihan Huang, Yue Wang, Jieyu Zhang, Letian Xue, Zihan Wang, Qineng Wang, Keshigeyan Chandrasegaran, Ruohan Zhang, Yejin Choi, et al. Theory of space: Can foundation models construct spatial beliefs through active exploration? In *The Fourteenth International Conference on Learning Representations*, 2026. 9, 41

[51] Julius Mayer, Mohamad Ballout, Serwan Jassim, Farbod Nosrat Nezami, and Elia Bruni. ivispar—an interactive visual-spatial reasoning benchmark for vlms. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 26745–26769, 2025. 9, 41

[52] Qiucheng Wu, Handong Zhao, Michael Saxon, Trung Bui, William Yang Wang, Yang Zhang, and Shiyu Chang. Vsp: Assessing the dual challenges of perception and reasoning in spatial planning tasks for vlms. *arXiv preprint arXiv:2407.01863*, 2024. 41

[53] Chan Hee Song, Valts Blukis, Jonathan Tremblay, Stephen Tyree, Yu Su, and Stan Birchfield. Robospatial: Teaching spatial understanding to 2d and 3d vision-language models for robotics. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 15768–15780, 2025. 9, 41

[54] R. Lang. *OSME 7 - Volume 4: Engineering Two*. OSME. Tarquin Group, 2021. 28

[55] Fanqing Meng, Jiaqi Liao, Xinyu Tan, Wenqi Shao, Quanfeng Lu, Kaipeng Zhang, Yu Cheng, Dianqi Li, Yu Qiao, and Ping Luo. Towards world simulator: Crafting physical commonsense-based benchmark for video generation. *arXiv preprint arXiv:2410.05363*, 2024. 41

[56] Qin Zhang, Peiyu Jing, Hong-Xing Yu, Fangqiang Ding, Fan Nie, Weimin Wang, Yilun Du, James Zou, Jiajun Wu, and Bing Shuai. Physion-eval: Evaluating physical realism in generated video via human reasoning. *arXiv preprint arXiv:2603.19607*, 2026. 41

[57] Hu Yue, Siyuan Huang, Yue Liao, Shengcong Chen, Pengfei Zhou, Liliang Chen, Maoqing Yao, and Guanghui Ren. Ewmbench: Evaluating scene, motion, and semantic quality in embodied world models. *arXiv preprint arXiv:2505.09694*, 2025. 41

[58] Yu Shang, Zhuohang Li, Yiding Ma, Weikang Su, Xin Jin, Ziyou Wang, Lei Jin, Xin Zhang, Yinzhou Tang, Haisheng Su, et al. Worldarena: A unified benchmark for evaluating perception and functional utility of embodied world models. *arXiv preprint arXiv:2602.08971*, 2026. 41

[59] Jiahan Zhang, Muqing Jiang, Nanru Dai, Taiming Lu, Arda Uzunoglu, Shunchi Zhang, Yana Wei, Jiahao Wang, Vishal M Patel, Paul Pu Liang, et al. World-in-world: World models in a closed-loop world. *arXiv preprint arXiv:2510.18135*, 2025. 41

[60] Chak-Wing Mak, Guanyu Zhu, Boyi Zhang, Hongji Li, Xiaowei Chi, Kevin Zhang, Yichen Wu, Yangfan He, Chun-Kai Fan, Wentao Lu, et al. Physicsmind: Sim and real mechanics benchmarking for physical reasoning and prediction in foundational vlms and world models. *arXiv preprint arXiv:2601.16007*, 2026. 41

[61] Zefan Cai, Haoyi Qiu, Tianyi Ma, Haozhe Zhao, Gengze Zhou, Kung-Hsiang Huang, Parisa Kordjamshidi, Minjia Zhang, Wen Xiao, Jiuxiang Gu, et al. Mmgr: Multi-modal generative reasoning. *arXiv preprint arXiv:2512.14691*, 2025. 41

[62] Harold Haodong Chen, Disen Lan, Wen-Jie Shu, Qingyang Liu, Zihan Wang, Sirui Chen, Wenkai Cheng, Kanghao Chen, Hongfei Zhang, Zixin Zhang, et al. Tivibench: Benchmarking think-in-video reasoning for video generative models. *arXiv preprint arXiv:2511.13704*, 2025. 41

[63] Jingqi Tong, Yurong Mou, Hangcheng Li, Mingzhe Li, Yongzhuo Yang, Ming Zhang, Qiguang Chen, Tianyi Liang, Xiaomeng Hu, Yining Zheng, et al. Thinking with video: Video generation as a promising multimodal reasoning paradigm. *arXiv preprint arXiv:2511.04570*, 2025. 41

[64] Qineng Wang, Wenlong Huang, Yu Zhou, Hang Yin, Tianwei Bao, Jianwen Lyu, Weiyu Liu, Ruohan Zhang, Jiajun Wu, Li Fei-Fei, et al. Enact: Evaluating embodied cognition with world modeling of egocentric interaction. *arXiv preprint arXiv:2511.20937*, 2025. 41

[65] Wei Chow, Jiageng Mao, Boyi Li, Daniel Seita, Vitor Guizilini, and Yue Wang. Physbench: Benchmarking and enhancing vision-language models for physical world understanding. *arXiv preprint arXiv:2501.16411*, 2025. 41

[66] Xinrun Xu, Pi Bu, Ye Wang, Börje F Karlsson, Ziming Wang, Tengtao Song, Qi Zhu, Jun Song, Zhiming Ding, and Bo Zheng. Deepphy: Benchmarking agentic vlms on physical reasoning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 34160–34168, 2026. 41

[67] Yuhao Wu, Maojia Song, Yihuai Lan, Lei Wang, Zhiqiang Hu, Yao Xiao, Heng Zhou, Weihua Zheng, Dylan Raharja, Soujanya Poria, et al. From perception to action: An interactive benchmark for vision reasoning. *arXiv preprint arXiv:2602.21015*, 2026. 41

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The claims in the Abstract and Introduction are supported by the benchmark construction in Section 2, the experimental results in Section 3, and the detailed benchmark and evaluation descriptions in the appendix.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: The paper discusses the main limitations of the benchmark design, evaluation protocol, and scope of the reported conclusions in Section 5.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

   Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: This paper does not contain theoretical results.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Data construction is described in Section 2; the main evaluation protocol is described in Section 3.1; task details, prompt templates, model snapshots, and inference settings are provided in Appendix A.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
  (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: The benchmark data, generation code, evaluation scripts, and reproduction instructions are prepared for release with the submitted artifacts, including the commands and environment needed to reproduce the main results.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (https://neurips.cc/public/guides/CodeSubmissionPolicy) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (https://neurips.cc/public/guides/CodeSubmissionPolicy) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The main experimental settings are presented in Section 3.1; model snapshots, prompt templates, decoding settings, and inference configurations are listed in Appendix B.2.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: The paper reports deterministic accuracies and success rates over fixed evaluation sets, together with sample counts and inter-annotator agreement for the human evaluation protocol.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: The paper reports the evaluated model configurations and inference setup in Section 3.1 and Appendix B.2, including the resources and execution settings needed to reproduce the benchmark runs.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics https://neurips.cc/public/EthicsGuidelines?

   Answer: [Yes]

   Justification: We have reviewed the NeurIPS Code of Ethics and confirm that this work conforms to it.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
   - The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [Yes]

Justification: The paper discusses the broader impact of the benchmark, including its intended use for diagnosing VLM limitations and the risks of over-interpreting benchmark performance, in Section 5.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [Yes]

Justification: The benchmark consists of synthetic topological reasoning and planning tasks, does not release a high-risk model, and does not contain personal, sensitive, or scraped human data; the paper therefore identifies no additional misuse safeguards beyond responsible release documentation.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Existing papers, model APIs, checkpoints, software dependencies, and templates are cited where used, and the released repository documentation records the corresponding versions, URLs, and usage terms.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The new benchmark is documented through the task catalog, environment descriptions, dataset statistics, prompt templates, and evaluation protocols in Appendix A and Appendix B.2.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [Yes]

Justification: Human evaluation used trained annotators under the same task instructions as the models. The annotation interface screenshots and agreement protocol are provided in Appendix B.1.1 and Appendix B.1.2. No external crowdworkers were employed.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [Yes]

Justification: Human evaluation involved trained annotators solving synthetic puzzle tasks and did not collect personal or sensitive data beyond task answers. The paper describes the annotation setting and explains why no external human-subject recruitment or additional IRB procedure was required.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
- We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
- For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

Answer: [Yes]

Justification: VLMs are evaluated as benchmark subjects; the evaluated model list, snapshots, and inference settings are provided in Section 3.1 and Appendix B.2. Additional LLM assistance for writing and literature review is disclosed in Appendix E.

Guidelines:

- The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
- Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.

Appendix

Table of Contents

A MINDTOPO: Benchmark Details

A.1 Cognitive-Science Grounding of the Five Properties

Our taxonomy is grounded in the developmental psychology account of topological cognition initiated by Piaget and Inhelder [1], refined by subsequent mathematical and educational analyses [10, 9], and formalised in standard algebraic topology [11].

Continuity. Continuity, in Piaget's phrasing, is the perception of a line, surface, or spatial field as an unbroken whole [1]. Poincaré captures its minimal form as a relation in which adjacent elements are indistinguishable, which is $A{=}B$, $B{=}C$. While distant ones are not, which is $A{\neq}C$. Piaget himself treats continuity as the synthesis of the other four primitives rather than a primitive on the same level, a view that Martin reinforces by showing the four to be mutually constitutive [10]. The standard mathematical formalisation is path-connectedness, the existence of a continuous map from $[0, 1]$ joining two points [11, Ch. 1]. Our Continuity tasks (2D MAZE, 3D MAZE, PIPE) ask whether two locations lie in the same path-component of a region given only its visual depiction.

Separation. Separation is the ability to distinguish neighbouring elements. Piaget illustrates the lack of it with the syncretic infant percept of an object leaning against a wall, perceived as a single ill-defined patch until separation analyses it into two units [1]. Martin argues that proximity and separation develop in tandem, finer separation does not displace proximity but allows the child to perceive different degrees of it over larger fields [10]. The topological correspondent is the decomposition of a space into its connected components, the most basic invariant preserved under homeomorphism [11, Ch. 0]. Our Separation tasks (IKEA, ONE-STROKE) ask the model to decide which substructures are independent units and which belong to a single connected whole.

Order. Order is the relation of spatial succession, by which elements are arranged one after another along a direction. Piaget documents it in the infant's gaze across the rungs of a cot and in the way young children seriate objects along a line [1]. Piaget treats seriation as one of the deep epistemic structures whose combination underwrites mathematical thought. And Martin notes that,

among Piaget's primitives, order admits the cleanest mathematical formalisation [10]. Its Euclidean refinement gives oriented coordinate systems and similarity transformations that preserve relative position. Our Order tasks test sequence read-out along a one-dimensional substrate (BEAD), point-tracking through ordered folds (ORIGAMI-POINT), and recovery of permutations under elementary operations (SWAP).

Enclosure. Enclosure, which Piaget calls *surrounding*, is the relation by which a boundary partitions space into an interior and an exterior [1]. In three dimensions it takes the form of *insideness*, exemplified by an object inside a closed box or by the contrast between a surface with a hole and one without. Algebraic topology formalises both sides of this intuition. The Jordan Curve Theorem [11, §2.B] shows that any subspace of S^2 homeomorphic to S^1 separates S^2 into two components, and a hole in a higher-dimensional space is detected by a spherical cycle bounding a missing interior [11, Ch. 2]. Our Enclosure tasks cover three faces of this property: SHEEP tests interior versus exterior judgement under fence-induced partition, HOLE tests the enumeration of bounded missing regions, and CHAT-NOIR tests planning under a closing-boundary constraint.

Knots. Knots occupy an ambiguous place in Piaget's own framework. Chapter 4 of *The Child's Conception of Space* uses them to probe surrounding, arguing that distinguishing a true knot from a deceptive one requires grasping how a strand encloses itself [1]. Strohecker later re-positions knot cognition as a comprehensive ability that draws on all of Piaget's substructures at once, together with set- and group-theoretic intuitions which are absent from the original primitives [11]. Recent empirical work supports this dissociation, showing that knot reasoning fails to track other physical-intuition abilities [13]. We therefore give Knots its own column, since the column captures phenomena such as crossing parity, linking number, and isotopy that no single Piaget's primitive accounts for. Our tasks KNOTS and UNTANGLE test the perceptual and interventional sides of this composite ability.

A.2 Task Catalog Overview

MINDTOPO comprises 13 task types organized by the five topological properties and two cognitive levels (Perception, Planning). Table 4 lists every task with its property slot, cognitive level, dataset size, and the appendix subsection in which it is described in detail. The remainder of this section (§A.3–§A.7) provides the per-task description.

Task	Property	Level	#Static Q	#Plan. inst.	Section
Maze-2D	Continuity	Perception	1,000	–	§A.3.1
Maze-3D	Continuity	Reasoning	1,000	–	§A.3.2
Pipe	Continuity	Planning	–	600	§A.3.3
IKEA	Separation	Reasoning	996	–	§A.4.1
One-Stroke	Separation	Planning	–	600	§A.4.2
Bead	Order	Perception	1,000	–	§A.5.1
Origami-Point	Order	Perception	1,008	–	§A.5.2
Swap	Order	Planning	–	600	§A.5.3
Sheep	Enclosure	Perception	1,005	–	§A.6.2
Hole	Enclosure	Reasoning	999	–	§A.6.1
Chat-Noir	Enclosure	Planning	–	600	§A.6.3
Knot	Knots	Perception	1,008	–	§A.7.1
Untangle	Knots	Planning	–	600	§A.7.2
Total			**8,016**	**3,000**	—

Table 4: MINDTOPO task catalog. The #Static Q column aggregates Perception and Reasoning instances (single-shot answer); #Plan. inst. counts planning episodes (multi-step rollouts). Each row points to the appendix subsection that describes its scene generator, difficulty tiers, and scoring rule.

A.3 Continuity Tasks

A.3.1 2D Maze (Continuity, Perception)

Targeted ability. The 2D Maze environment isolates a model's ability to reason about *global connectivity* purely from a top-down rendering, with no symbolic graph input.

Task formulation. We instantiate two question types over the same scene generator. **Q1 (`reachability_set`)** samples 3–5 labeled points and asks which others are connected to target A, with the answer returned as a bracketed name list (e.g. `[B, D]`). **Q2 (`bar_removal`)** fixes two points A, B that are guaranteed disconnected in the base maze, then paints N colored bars over a subset of the blocking walls; the model must list every bar whose single-bar removal reconnects A and B (e.g. `[purple, red]`). Both share `answer_type = name_list` and are scored by set equality.

Scene generation. Each scene is produced in three deterministic, seed-driven stages, with ground truth supplied by a connectivity oracle.

Skeleton (DFS). Recursive-backtracking DFS on the $N \times N$ grid yields a spanning tree, then $K = \mathrm{round}(\rho\, M)$ of the $M = 2N(N-1)$ internal walls are closed/opened to hit the target wall density ρ.

Wall taxonomy. A fraction `diagonal_ratio` of closed walls is converted into full diagonal walls (Mode-dA); a fraction `partial_ratio` is further demoted to half-length variants (Mode-B/C for horizontal/vertical edges, Mode-dB/dC for diagonals). Setting the side length of each grid cell to 1 without loss of generality, the renderer admits five wall types organized into three families (Table 5). A *horizontal/vertical wall* (**Mode-A**) coincides with the entire shared edge between two axis-adjacent cells, with length 1; it is the canonical closure used by classical maze generators and induces the standard 4-neighbor cell graph. A *diagonal wall* (**Mode-dA**) is embedded *inside* a single cell rather than on its boundary, spanning one of the two cell diagonals (slash or backslash) with length $\sqrt{2}$; partitioning each cell into four micro-triangles meeting at the cell center, a Mode-dA wall closes the shared edge between two micro-triangles, separating the cell interior into two disjoint halves. A *partial wall* has the same orientation and host edge as one of the full walls above but spans only half its length, and is rendered with the same line style: **Mode-B** is centered on a horizontal/vertical edge while **Mode-C** is anchored at one of its endpoints (both length $1/2$); **Mode-dB** is centered at the cell center while **Mode-dC** is anchored at one of the diagonal's endpoints (both length $\sqrt{2}/2$). Because every partial wall leaves an unobstructed sub-segment of its host edge, it does not separate the two regions adjacent to that edge in the underlying maze graph.

Annotation. Q1 samples points uniformly from diagonal-free cells under a min-pairwise-distance constraint; Q2 additionally paints N colored bars on blocking walls (Mode-A horizontal/vertical or Mode-dA diagonal) and rejects the scene unless A, B are disconnected and at least one bar's single removal reconnects them.

Connectivity oracle. Ground truth is computed by a BFS over a *4-triangle decomposition* of each cell (T_N, T_E, T_S, T_W sharing the cell center). Each horizontal/vertical edge is owned by one triangle and each cell-internal diagonal separates exactly two of them, so a single BFS uniformly handles axial walls and diagonal walls. Because labeled points sit at cell centers (the shared vertex of all four triangles), the start cell's four triangles are all seeded at hop 0. The same routine drives both the answer key and the ORACLE sanity baseline (which by construction achieves $\mathrm{acc.} = 1.000$).

Table 5: **Wall taxonomy of the 2D Maze renderer.** Five wall types organized by geometric embedding, assuming unit cell edge length. "Has passage?" indicates whether the two regions adjacent to the wall remain connected in the underlying maze graph.

Wall type	Has passage?	Geometry (cell edge length $= 1$)
Mode-A	$\times$	full horizontal/vertical edge, length 1
Mode-B	✓	centered half-edge, length $1/2$
Mode-C	✓	endpoint-anchored half-edge, length $1/2$
Mode-dA	$\times$	full cell diagonal, length $\sqrt{2}$
Mode-dB / dC	✓	partial cell diagonal, length $\sqrt{2}/2$

Parameter	Easy	Medium	Hard
`grid_size`	4	5	6
`wall_density` (jitter range)	0.35–0.45	0.40–0.50	0.45–0.55
`diagonal_ratio`	0	0	0.25
`partial_ratio`	0	0.30	0.30
Q1 `min_pairwise_distance`	0	3	3
Q2 `min_pairwise_distance`	0	3	5
Q2 `min_correct_removals`	1	2	2
Q2 `min_area_fraction`	0	$1/4$	$1/3$

Table 6: **Difficulty tiers for the 2D Maze environment.** The lower block lists per-tier rejection-sampling constraints that suppress trivially-solvable scenes: pairs that are too close, single-answer Q2 instances (guessable at $1/N$), and lopsided components in which the smaller side's surrounding bars dominate the correct set.

Difficulty tiers. Difficulty is determined directly by the generation configuration rather than by a post-hoc score. Easy/Medium/Hard differ in grid size, wall density, and which wall shapes are admitted; Medium and Hard additionally apply rejection-sampling constraints that exclude trivially-solvable scenes (Table 6).

Output format and scoring. Both Q1 and Q2 share the `name_list` answer contract: the prompt instructs the model to return JSON only, with schema $\{$`"answer"`$:[\langle$*name*$\rangle, \ldots]\}$ and `[]` reserved for the empty case. The legal vocabulary is sample-specific — Q1 is restricted to the scene's labeled point names and Q2 to its bar colors — and is registered as a `NAME_LIST:` spec consumed by a layered parser (`topobench_eval.answer_parser`) that first attempts a strict `{"answer": ...}` JSON extraction and only falls back to final-answer phrase matching, in-text legal-value scanning, and a tail-of-text token as successive recoveries. Predictions and ground truth are then coerced to uppercased, whitespace-stripped frozensets and compared by *set equality*, so order, case, and duplicates do not affect the score.

We aggregate three quantities over the evaluation split: **pairwise accuracy** (mean per-scene $0/1$ correctness), **JSON parse rate** (fraction of responses recovered by the strict-JSON layer), and **answer-format hit rate** (fraction whose strict JSON additionally passes the per-scene shape and legal-value validator). The latter two isolate format-following failures from reasoning failures.

A.3.2 3D Maze (Continuity, Reasoning)

Targeted ability. This task evaluates whether a model can infer global connectivity in a metric 3D environment from visual evidence alone. Unlike the 2D Maze task, the scene contains furnished rooms, occlusions, and door states, so the model must integrate multiple views before deciding whether two marked locations belong to the same connected component.

Task formulation. Each instance provides five rendered views of the same ProcTHOR house: a top-down view and four oblique views from fixed cardinal directions. Labeled navigation points are placed on valid floor locations. We use two reasoning question types. In the target-point connectivity question, the model is given a source point and must list every other labeled point reachable from it. In the mutually connected subset question, the model selects the unique option whose listed points are all mutually connected under the shown door states.

Scene generation. We instantiate furnished indoor layouts with the ProcTHOR/AI2-THOR renderer and treat each house as a 3D maze over valid navigation locations. For each sampled scene, the generator fixes the requested number of rooms, controllable doors, and labeled points, then samples door states and point locations with a deterministic seed. The five views are rendered from the same state and stored with the question row. Ground truth is computed by a navigation connectivity oracle that respects walls, closed doors, furniture, and other solid obstacles, while open doors and open floor or corridor spaces remain passable.

Difficulty tiers. Difficulty is derived from the structural complexity of the generated house rather than assigned by a separate argument. The score adds the number of rooms, the number of openable doors, and twice the number of labeled points beyond the first two; scores at most 7 are Easy, scores from 8 to 13 are Medium, and scores at least 14 are Hard. The current split contains 500 rendered houses and 1,000 reasoning questions, with one target-point connectivity question and one mutually connected subset question per house.

Difficulty	Rooms	Doors	Points	Scenes	Questions
Easy	2–3	1	3	150	300
Medium	4	2–3	3–4	200	400
Hard	7	5	3–4	150	300

Table 7: Difficulty tiers for the 3D Maze task.

Output format and scoring. For target-point connectivity questions, the expected answer is a JSON list of point names, e.g., `{"answer":["B","D"]}`, and the empty set is represented as `{"answer":[]}`. For mutually connected subset questions, the expected answer is the selected option label, e.g., `{"answer":"2"}`. Predictions for target-point connectivity questions are scored by exact set equality after normalization, while multiple-choice predictions are scored by exact option match.

A.3.3 Pipe (Continuity, Planning)

Targeted ability. Pipe Rotation evaluates sequential continuity reasoning under local rotations. A successful model must identify disconnected pipe components, anticipate how 90-degree rotations change openings, and plan a sequence that connects every pipe segment to the source.

Task formulation. The model observes a square pipe board at each step. The green source marks the root of the network; connected pipes are rendered in green and disconnected pipes in blue. At every turn, the model chooses one non-empty cell and the environment rotates that pipe clockwise by 90°. An episode succeeds when all non-empty pipe cells are connected to the source through matching openings before the action budget is exhausted.

Scene generation. Each puzzle is generated from a connected tree-shaped pipe network. The generator first samples a square grid, source cell, active pipe count, and number of three-way junctions, rejects loops and four-way junctions, and then scrambles each pipe by seeded random rotations. This construction preserves a known solved state while requiring the model to recover it through local rotations from the rendered board.

Difficulty tiers. Difficulty controls grid size, network density, branching, and oracle solution length. The benchmark uses 60 episodes per tier and records the oracle rotation count together with an action budget equal to a slackened multiple of the solution length.

Difficulty	Grid	Active pipes	Junctions	Oracle rotations	Budget
Easy	4×4	11–14	2–4	10–20	13–26
Medium	5×5	18–23	4–7	17–33	23–43
Hard	5×5	23–25	7–9	23–37	30–49

Table 8: Difficulty tiers for the Pipe Rotation task. Each tier contains 60 planning episodes.

Output format and scoring. The action contract is `{"answer":{"x": <column>, "y": <row>}}`, where columns and rows follow the visual labels on the board. We evaluate the full trajectory rather than individual moves: an episode is correct if the environment reaches a state in which every non-empty pipe belongs to the source-connected component within the prescribed step budget.

A.4 Separation Tasks

A.4.1 IKEA (Separation, Reasoning)

Targeted ability. IKEA tests whether a model can recover object decomposition from a rendered 3D assembly. The task targets separation reasoning: the complete object must be mentally partitioned into valid subassemblies, while distractors preserve plausible shape and category cues.

Task formulation. Each instance contains one image of a complete object and five candidate decomposition options. The complete-object image combines two oblique views, while each option shows a proposed split into two subassemblies. The model must select the option that exactly matches a valid decomposition of the original object.

Scene generation. We build the task from a catalog of IKEA-style objects with known primitive parts and object categories. For each object, the generator selects a valid two-part split as the correct option and constructs four structured distractors: same-category component replacements, missing-component variants, and extra-component variants sampled from different target partitions. The five options are shuffled deterministically, yielding one complete-object rendering and five option renderings per question.

Difficulty tiers. Difficulty is defined by the primitive part count of the complete object. The current split contains 996 reasoning questions from 83 eligible objects and five object categories. Each eligible object is sampled with repeated seeds so that option ordering and distractor selection vary while the underlying decomposition remains exact.

Difficulty	Part count	Object categories	Questions
Easy	3–5	Bench, Chair, Table	360
Medium	6–8	Chair, Misc, Table	432
Hard	9–12	Chair, Desk, Misc, Table	204

Table 9: Difficulty tiers for the IKEA separation-reasoning task.

Output format and scoring. The canonical response is a multiple-choice JSON answer, e.g., `{"answer":"E"}`. Predictions are normalized to the option letter and scored by exact match against the shuffled correct option.

A.4.2 One-Stroke (Separation, Planning)

Targeted ability. One-Stroke evaluates separation-aware path planning. The model must draw a single continuous stroke that separates differently colored regions while keeping cells of the same color connected on the same side of the stroke.

Task formulation. The environment presents a colored grid with a cursor starting at the bottom-left corner and a target at the top-right corner. At every turn, the model issues one of four moves, `U`, `D`, `L`, or `R`. The stroke cannot leave the board or reuse an invalid edge, and the episode succeeds only when the completed path reaches the target while satisfying the color-region separation constraint.

Scene generation. Each puzzle is produced by first sampling a legal no-loop construction path, then filling region-safe color blocks induced by that path. The generator recomputes the shortest valid solution with BFS and keeps only instances whose shortest solution length falls inside the tier range. The stored oracle is therefore the shortest path for the accepted puzzle rather than the initially sampled construction path.

Difficulty tiers. Difficulty is controlled by board size, number of colors, and shortest solution length. The benchmark contains 60 planning episodes per tier.

Difficulty	Board	Vertex grid	Colors	Shortest solution	Budget
Easy	4×4	5×5	3	10–14	12–20
Medium	5×5	6×6	3–4	12–18	17–27
Hard	6×6	7×7	5	20	24–34

Table 10: Difficulty tiers for the One-Stroke planning task.

Output format and scoring. The action contract is `{"answer":"U"}` with the answer drawn from `{U,D,L,R}`. We score the full trajectory: an episode is successful if the model reaches the target within the action budget and the final stroke satisfies the same-color grouping and different-color separation constraints.

A.5 Order Tasks

A.5.1 Bead String (Order, Perception)

Targeted ability. Bead String isolates a model's ability to read a colour ordering along a one-dimensional substrate that has been embedded into a non-trivial 3D curve. The task's identity is given by colour, and the only structure to recover is the sequence in which colours are encountered as the string is traversed end to end.

Task formulation. Each instance shows a 3D scene in which a string of coloured beads is threaded along a sampled space curve and rendered from one or two viewpoints. Two question types are exposed over the same scene. The describe-sequence question (`T_BS01`) asks the model to list every bead colour in traversal order as a comma-separated list (e.g. `"RED, BLUE, GREEN"`). The pair-relationship question (`T_BS02`) asks how two highlighted beads are positioned along the string (direct neighbours, separated by one intermediate bead, or further apart) and is returned as a single label.

Scene generation. The renderer is implemented as a Vite + Playwright pipeline. For each instance the generator samples a curve family (straight, arc, S-curve, helix, wiggly), discretises it into bead positions under a fixed inter-bead gap, assigns colours from an eight-element palette without replacement, and renders the scene at 1024×1024 from canonical isometric and front-facing cameras. Ground truth is the colour sequence read off the generated curve, so it is exact and reproducible from a deterministic seed.

Difficulty tiers. Difficulty is controlled jointly by bead count and curve complexity (Table 11). The current split contains $1{,}000$ perception questions, with the released distribution skewed towards the Easy tier to anchor the lower end of the curve.

Difficulty	Bead count	Curve types	Questions
Easy	3–5	straight, arc	390
Medium	5–7	S-curve, helix	311
Hard	7–10	wiggly, helix	299

Table 11: Difficulty tiers for the Bead String task.

Output format and scoring. For describe-sequence questions, the canonical response is JSON of the form `{"answer":"RED, BLUE, GREEN"}`; the parser canonicalises the response into an ordered list of upper-cased colour tokens drawn from the eight-name palette, and a prediction is correct only if both order and multiplicity match the ground truth exactly. The pair-relationship question returns a single label and is scored by exact match after upper-casing.

A.5.2 Origami Point Tracking (Order, Perception)

Targeted ability. Under Piaget's framework of conservation under transformation [1], this task tests whether current multimodal large language models are capable of *distinguishing the invariant* (the identity of marked points on a piece of paper) from *the variant* (their Euclidean location in 3D space) as the paper is folded.

Task formulation. We modify the Origami Simulator [54] to predefine points on the paper and to progress through a preset rotation trajectory. We instantiate the task across four origami bases (bird, waterbomb, pinwheel, open-sink) and three difficulty tiers, producing 84 instances per (model, tier) cell for a total of 1008 instances. Each instance is a sequence of N rendered images of an origami model, morphing from a flat sheet to a folded-and-rotated state (step N), with K labeled points (two always-visible anchors and one to four points revealed at the final state). We label the points alphabetically and reveal these labels at the final state. In our paper, we use a forward progression (flat to folded) for evaluation.

Scene generation. For each (model, difficulty) cell, we generate trajectories by searching over sequences of (fold percentage, camera viewpoint, model rotation) tuples. A trajectory is accepted only if both anchor points remain geometrically visible at every step and all hidden points become visible at the final step. Visibility is computed in closed form: each labeled point lies at a fixed barycentric coordinate on a mesh face, and a ray cast from the point to the camera is tested against the full mesh, since folded panels block sight from either side. After acceptance, each point's barycentric position is refined on a small grid to maximize visibility, and the sequence is replayed end to end in the simulator to confirm the refined placements. Every label is a geometric property of the simulated mesh rather than a human annotation, so ground truth is exact and reproducible from a fixed seed.

Difficulty tiers. We define difficulty along two axes: peak rotation magnitude and whether the rotation animates across the sequence. The Easy tier presents an origami model at a constant pose with no inter-step rotation, with only one side visible throughout. The Medium tier progressively ramps from a flat hero-shot view to a peak magnitude, with only one side of the paper visible throughout. The Hard tier doubles the rotation strength and additionally places points on the underside of the paper, requiring the model to reason about points that become visible only after rotation. Per-tier parameters are summarized in Table 12.

Parameter	Easy	Medium	Hard
Initial Points	2	2	2
Total Points	3–6	3–6	3–6
Back-side reveals	no	no	yes
Peak yaw (rad)	0.5	0.5	1.0
Step Count	5	10	10

Table 12: Difficulty tiers for the Origami Point Tracking task.

Output format and scoring. The expected response is a JSON list of upper-case letters drawn from the instance-specific label set, e.g. `{"answer":["A","C"]}`; the empty case is `{"answer":[]}`. Predictions and ground truth are coerced into frozensets and compared by set equality, so the answer is order-insensitive but identity-sensitive. Every emitted letter must correspond to one of the unmarked dots shown on the flat sheet, and no spurious letters are allowed.

A.5.3 Swap Puzzle (Order, Planning)

Targeted ability. Swap Puzzle evaluates whether a model can reason about order through a sequence of constrained permutations. The model must transform an initial row into a target row by using the blank slot as the only exchange medium.

Task formulation. Each episode provides two images at every turn: the current arrangement and the goal arrangement. The row contains a blank slot and N colored blocks. At each step, the model

selects one non-empty slot; the chosen block is swapped with the blank. The task is solved when the current arrangement exactly matches the target arrangement.

Scene generation. For each block count, the generator samples an initial arrangement and a goal arrangement under a deterministic seed, then computes the exact shortest-path distance in the state graph where any non-empty slot may swap with the blank. The benchmark retains this theoretical minimum step count and sets a step budget by multiplying it by 1.5 and rounding up.

Difficulty tiers. Difficulty is determined by the number of colored blocks. The current split contains 30 episodes for each block count from 2 to 7, yielding 60 planning episodes per tier.

Difficulty	Blocks	Episodes	Shortest distance	Budget
Easy	2–3	60	1–4	2–6
Medium	4–5	60	1–7	2–11
Hard	6–7	60	4–8	6–12

Table 13: Difficulty tiers for the Swap Puzzle task.

Output format and scoring. The canonical action is a slot index in JSON form, e.g., {"answer":2}. We evaluate the executed episode, not the textual plan: the prediction is correct only if the model reaches the goal arrangement within the step budget.

A.6 Enclosure Tasks

A.6.1 **Hole** (Enclosure, Reasoning)

Targeted ability. Hole evaluates enclosure reasoning over solid 3D objects. The model must count only through-holes that connect the top surface to open space, while ignoring pits, shadows, and shallow depressions that do not pass through the board.

Task formulation. Each instance shows a single top-down rendering of a procedurally generated board. The prompt asks for the number of visible holes on the top board. If a lower board is visible in the scene, the model must ignore it and evaluate only the top board.

Scene generation. The renderer samples a board shape from rectangles, circles, and polygons, then places openings and distractor depressions according to the selected difficulty. Ground truth is generated directly from the procedural layout: only openings that pass through the board to open space contribute to the answer.

Difficulty tiers. Difficulty controls both the target answer range and the set of distractor hole types. The current split contains 999 reasoning questions, with 333 questions per tier and one rendered image per question. Table 14 reports both the generator's target visible-hole range and the observed ground-truth range in the current data split.

Difficulty	Target holes	Observed GT	Board shapes	Questions
Easy	3–7	3–7	rect, circle, polygon	333
Medium	4–9	3–9	rect, circle, polygon	333
Hard	5–12	5–14	rect, circle, polygon	333

Table 14: Difficulty tiers for the Hole reasoning task.

Output format and scoring. The expected answer is a scalar integer, e.g., `{"answer":6}`. Predictions are scored by exact integer match after parsing; off-by-one counts and counts that include non-through pits are marked incorrect.

A.6.2 Enclosure-Sheep (Enclosure, Perception)

Targeted ability. Enclosure-Sheep probes inside-versus-outside reasoning under an arbitrary, possibly piece-wise fence. The model must recover the planar partition induced by the fence and decide region membership for each labelled sheep, the canonical Jordan-curve-style enclosure judgement [11, §2.B].

Task formulation. Each instance shows a top-down render of a pasture in which numbered sheep are scattered across a fence layout. Four question types share the same scene. *Count-inside* asks how many sheep lie inside the enclosure; *escape-possibility* asks whether a designated sheep can reach the unbounded exterior without crossing a fence segment; *max-cell-count* asks for the population of the largest enclosed cell; and *fence-repair* asks which broken segment, when patched, traps the largest number of currently-loose sheep. Counting answers are non-negative integers; the remaining types use single upper-case option letters.

Scene generation. Scenes are produced by a Vite and Playwright renderer that supports two layout families, *single fence* (one closed boundary partitioning the plane into one inside and one outside) and *partitioned* (a fence with internal walls subdividing the inside into multiple cells). For each pair, the generator samples fence shape, gap positions, and sheep coordinates from a deterministic seed, runs a polygon-membership oracle to produce ground truth, and renders the scene at the canonical resolution. The scene is rejected and re-sampled if the answer key is degenerate. For example, all sheep on one side, or zero sheep inside.

Difficulty tiers. Difficulty controls fence complexity, the number of partitioned cells, and the sheep count (Table 15). The current split contains 1,005 perception questions, balanced across the two layout families with 335 questions per tier.

Difficulty	Layout families	Question types	Questions
Easy	fence, partitioned	all four	335
Medium	fence, partitioned	all four	335
Hard	fence, partitioned	all four	335

Table 15: Difficulty tiers for the Enclosure-Sheep task.

Output format and scoring. The canonical responses are `{"answer":<int>}` for counting questions and `{"answer":"<letter>"}` for the remaining types. After whitespace stripping and upper-casing, predictions are compared to ground truth by exact match; off-by-one counts and out-of-vocabulary letters are marked incorrect.

A.6.3 Chat Noir (Enclosure, Planning)

Targeted ability. Chat Noir evaluates adversarial enclosure planning. The model must progressively block a hex-grid board so that the moving cat loses every path to the boundary before it can escape.

Task formulation. At each turn, the model observes the current board and selects one open non-cat cell to block. The cat then moves according to the tier's policy. The episode succeeds if the cat has no remaining path from its current cell to any boundary cell, and fails if the cat reaches the boundary or the action budget is exhausted.

Scene generation. The generator samples board radius, cat position, and initial blockers from a deterministic seed. Initial blockers are drawn uniformly from non-cat cells, then a bounded search filter rejects setups in which the cat is already trapped, has no legal move, lacks a path to the boundary, or has too few winning first block actions. This produces episodes that are solvable but still require multi-step enclosure planning.

Difficulty tiers. Difficulty is defined by the cat policy rather than by board radius. Easy uses a mixed walker that follows a shortest path with probability 0.3 and otherwise samples a legal adjacent

move uniformly. Medium uses greedy shortest-path movement. Hard uses a connectivity-aware greedy policy that avoids immediate one-move traps when possible. The current split contains 180 planning episodes.

Difficulty	Cat policy	Radius	Initial blockers	Budget
Easy	mixed random/greedy	3–4	6–13	28–50
Medium	greedy shortest path	3–4	8–16	26–47
Hard	connectivity-aware greedy	3–4	10–20	24–44

Table 16: Difficulty tiers for the Chat Noir planning task.

Output format and scoring. The canonical action is a blocked cell index, e.g., `{"answer":12}`. We score the resulting interaction: an episode is correct only if the model traps the cat before escape and within the maximum number of allowed actions.

A.7 Knots Tasks

A.7.1 **Knots** (Knots, Perception)

Targeted ability. Knots task evaluates whether a model can classify the global topological type of one or more rendered curves. The task targets the core invariants of knot theory. It distinguishes an unknotted ring from a genuinely knotted one, an isolated component from a non-trivial link, and recovers the crossing number of more complex configurations.

Task formulation. Each instance presents two rendered views of a 3D rope configuration, which is a canonical isometric view paired with a complementary front view. The primary structure-classification question (`T01`) is a five-way multiple choice with options A: simple ring, B: knot, C: link, D: unlinked multiple, E: other; answers are returned as a single upper-case letter. Auxiliary question types reuse the same renders to probe finer-grained structure, which is crossing-number estimation, named-knot identification, link-component count, and equivalence classification across paired scenes.

Scene generation. The renderer instantiates ropes from a catalog of 15 knot families covering torus knots, twist knots, simple links, and unlinked multi-rope configurations, with viewpoint, rope colour, and crossing decoration sampled from a deterministic seed. The two views are chosen so that the structure-classification answer is decidable from the pair without disambiguating ambiguous projections; in the harder tier the views are deliberately taken near classification boundaries to suppress trivial visual shortcuts.

Difficulty tiers. Difficulty trades off topology complexity like crossing count, knot family against visual difficulty like viewpoint ambiguity, rope-colour similarity. The current split contains 1,008 reasoning questions in total (Table 17).

Difficulty	Topology load	Visual trap	Questions
Easy	low crossings, single rope	none	358
Medium	moderate crossings	viewpoint	422
Hard	high crossings, multi-rope	viewpoint+colour	228

Table 17: Difficulty tiers for the Knot Detection task.

A.7.2 **Untangle** (Knots, Planning)

Targeted ability. Untangle task evaluates step-by-step rope-untangling under a constrained action space. The model must reduce the number of rope-rope crossings to zero by repeatedly relocating rope endpoints to legal grid holes, without lifting strands across one another. Success requires reasoning over the present crossing structure as well as the topological consequence of every candidate move.

Task formulation. The environment exposes a top-down view of a $G \times G$ grid of holes through which R coloured ropes are threaded, with each rope's two endpoints occupying distinct holes. At every turn the model observes the current rendered grid and emits an action that picks up one endpoint and moves it to an unoccupied target hole; the rope is re-threaded along a deterministic path through the grid interior. An episode succeeds when the rendered configuration contains zero logical crossings before the action budget is exhausted. Illegal moves consume budget without altering the state, such as source hole not an endpoint and target hole occupied.

Scene generation. For each tier the generator first instantiates a known unknotted configuration, then applies a fixed number of seeded scramble moves drawn from the same endpoint-relocation operator. This guarantees that every released episode is solvable within the scramble depth, which doubles as the maximum oracle plan length recorded with the instance. Ground truth is computed by an A*-style oracle search over rope states, with the unknot (logicalCrossings $= 0$) as goal and the scramble inverse as a length-optimal solution.

Difficulty tiers. Difficulty scales the grid dimension, the rope count, and the scramble depth (Table 18). All tiers share the same step budget of 15 actions, and each contains 200 episodes for a total of 600 planning instances.

Difficulty	Grid	Holes	Ropes	Action budget	Episodes
Easy	5×5	25	4	15	200
Medium	6×6	36	5	15	200
Hard	6×6	36	6	15	200

Table 18: Difficulty tiers for the Knots Untangle task.

A.8 Dataset Statistics

Property	Static #Q	Plan. inst.	Easy	Medium	Hard	Total
Continuity	2,000	600	833	933	834	2,600
Separation	996	600	560	632	404	1,596
Order	2,008	600	926	847	835	2,608
Enclosure	2,004	600	868	868	868	2,604
Knots	1,008	600	558	622	428	1,608
Total	**8,016**	**3,000**	**3,745**	**3,902**	**3,369**	**11,016**

Table 19: MINDTOPO dataset composition across the five topological properties, the Reasoning and Planning levels, and the three difficulty tiers. Planning instances are distributed uniformly across tiers.

A.9 Evaluation Design

Reasoning tasks. Reasoning tasks share a single-shot answer contract. The model receives one rendered scene or a fixed bundle of views, emits one JSON response, and is scored without rollout. Across the eleven static tasks the answer types fall into four families: *set* (2D MAZE Q1/Q2, 3D MAZE connectivity, ORIGAMI POINT TRACKING), *sequence* (BEAD STRING describe-sequence), *scalar integer* (HOLE, ENCLOSURE-SHEEP count, KNOT DETECTION crossing), and *multiple-choice letter* (IKEA, KNOT DETECTION T01, BEAD STRING pair-relationship, ENCLOSURE-SHEEP non-counting types). All four families are parsed by the same layered extractor in `topobench_eval.answer_parser`, which first attempts a strict `{"answer": ...}` JSON extraction and successively falls back to final-answer phrase matching, in-text legal-value scanning, and tail-of-text token recovery before declaring a parse failure. Set answers are scored by frozenset equality, sequences by ordered list equality, scalars by exact integer match, and letters by exact match against the per-instance legal vocabulary.

Planning tasks. Planning tasks expose an action interface and are scored on the executed trajectory rather than on the textual plan. At every turn the model receives the current rendered state and emits one JSON action; the environment validates legality, applies the action or charges the budget for an illegal one, and renders the next state. Each task ships with a per-instance step budget computed by slackening its oracle solution length: $1.3\times$ for PIPE, $1.5\times$ for SWAP PUZZLE and ONE-STROKE, a fixed 15-action cap for KNOTS UNTANGLE, and a tier-specific table for CHAT NOIR. There is no retry: budget exhaustion or violation of the success predicate ends the episode and counts as a failure. We report two reference baselines per planning task: the ORACLE baseline (the precomputed optimal plan; achieves $\mathrm{acc.} = 1.000$ by construction) and the RANDOM baseline (a uniform legal-action policy under the same budget).

Metrics. The primary per-task metric is *accuracy*: per-instance $0/1$ correctness averaged over the evaluation split. Static tasks also report *JSON parse rate* (fraction recovered by the strict-JSON layer) and *answer-format hit rate* (fraction whose strict JSON additionally passes the per-instance shape and legal-value validator), isolating format-following failures from reasoning failures. Planning tasks additionally report *mean over-optimality*, the average gap between the executed trajectory length and the oracle plan length on solved episodes. We aggregate accuracy in two ways: a *per-level mean* which is separate averages for Reasoning and Planning and a *macro-property mean* which is the mean of the five property-wise accuracies, weighting each topological invariant equally regardless of its question count.

Summary. The unified pipeline therefore handles three sources of answer non-uniqueness in a consistent way: *set-valued* answers are coerced to frozensets so that order, case, and duplicates do not inflate or deflate the score; *multi-valid* planning trajectories are accepted as long as the final-state predicate holds within the budget, so any plan that unties the knot or traps the cat is correct; and *length-mismatched* sequences (e.g. a bead-colour list of the wrong length) fall through the strict equality check and are marked incorrect, rather than being partially credited.

B Experiments and Analysis

B.1 Human Evaluation

B.1.1 Annotation Interface and Human Performance Evaluation

Human annotators use the same visual inputs and answer schemas as model evaluations. For reasoning tasks, the interface loads each JSONL sample, renders the image panel(s), exposes a task-specific answer widget (single choice, integer field, set/list entry, or sequence entry), and saves an optional comment field for ambiguous cases. For planning tasks, annotators play the same browser-based environment as the model runner: the system resets the episode from the JSONL `reset_config`, records every selected action, and scores success from the environment terminal state rather than from self-reported answers.

The reported human row in Table 1 is computed with the same parser and scorer used for models. Set-valued answers are canonicalized by sorting and de-duplicating labels, scalar answers are normalized through the shared answer parser, and planning episodes are marked correct only when the environment terminates successfully. The human calibration covers the same stratified difficulty tiers as the benchmark and yields near-ceiling performance on most tasks: 97.18 on 2D Maze, 98.62 on 3D Maze, 94.37 on IKEA, 95.77 on Bead, 91.94 on Origami Point, 99.06 on Sheep, 97.65 on Hole, and 91.55 on Knots, with 100.00 on all five planning tasks in the calibration set. These numbers should be interpreted as an empirical human-upper-bound check, not as additional training supervision: ground-truth labels remain generator-derived.

B.1.2 Inter-Annotator Agreement

We measure inter-annotator agreement (IAA) on a subset of the benchmark before any adjudication. The subset is annotated independently by three trained annotators under the same instructions as the models. We report raw percent agreement after applying the same answer canonicalization used by the benchmark scorer. The full-agreement IAA is 0.90, indicating high consistency among annotators.

For reasoning tasks, agreement is computed by exact match after answer canonicalization. For planning tasks, agreement is computed under the task's deterministic evaluation protocol: two annotations agree only when they reach the same evaluated outcome. Disagreements are not resolved by majority vote for the purpose of IAA calculation. Instead, they are reviewed separately to identify potential ambiguity, interface issues, or annotation errors.

B.2 Models, Prompts, and Setup

Evaluation protocol. Static reasoning tasks are evaluated as single-turn visual QA with `temperature=0`, `do_sample=false`, and a maximum of 2048 newly generated tokens. Each prompt asks for exactly one JSON object, and the evaluator applies the shared fallback parser only after the model has finished. Planning tasks use the same deterministic decoding settings and run through the Hydra+Playwright environment runner; each turn includes the current rendered observation, the task rules, and the legal answer schema. We do not give symbolic state graphs, oracle connectivity, or hidden generator metadata to any model. Failed API calls are retried according to the per-provider configuration, after which the episode or sample is marked invalid.

All static images are supplied at the renderer's native resolution and preserved as PNG inputs. Planning screenshots are captured from the browser frontend; environments that need a fixed viewport use the configured 1440×960 or 1440×1080 viewport before scene-only cropping. Every task uses one prompt template per property/task family, fixed across models. We use no task-specific prompt tuning after seeing model outputs.

Organization	Model Name	Snapshot	Full Name	Evaluation Pipeline
Proprietary Models				
OpenAI	GPT-5.5	2026-05 eval	`gpt-5.5`	Static + planning
OpenAI	GPT-5.4 mini	2026-03-17	`gpt-5.4-mini-2026-03-17`	Static + planning
Google	Gemini 3.1 Pro	2026-05 eval	`gemini-3.1-pro-preview`	Static + planning
Google	Gemini 3.1 Flash Lite	2026-05 eval	`gemini-3.1-flash-lite-preview`	Static + planning
Open-Weight Models				
InternLM	InternVL-3.5	checkpoint	`internvl3.5-241b-a28b`	Static + planning
NVIDIA	Nemotron Nano 12B v2 VL	NIM snapshot	`nvidia/nemotron-nano-12b-v2-vl`	Static + planning
Google	Gemma-4-31B-IT	private API snapshot	`google/gemma-4-31b-it`	Static + planning
Alibaba	Qwen3.5-VL-397B	NIM snapshot	`qwen/qwen3.5-397b-a17b`	Static + planning
NVIDIA	Cosmos-Reason2	self-hosted snapshot	`nvidia/Cosmos-Reason2-8B`	Static + planning
ByteDance	BAGEL-7B	self-hosted snapshot	`ByteDance-Seed/BAGEL-7B-MoT`	Static + planning
ThinkMorph	ThinkMorph-7B	self-hosted snapshot	`ThinkMorph/ThinkMorph-7B`	Static + planning
Image Generative Models				
OpenAI	GPT-Image-2	2026-05 eval	`gpt-image-2`	Image-edited imagined rollouts
ByteDance	BAGEL-7B	self-hosted snapshot	`ByteDance-Seed/BAGEL-7B-MoT`	Image imagined rollouts
ThinkMorph	ThinkMorph-7B	self-hosted snapshot	`ThinkMorph/ThinkMorph-7B`	Image imagined rollouts
Video Generative Models				
Wan-AI	Wan2.2-I2V-A14B	hosted snapshot	`Wan-AI/Wan2.2-I2V-A14B`	Image-to-video rollouts
THUDM	CogVideoX-5B / I2V	self-hosted snapshot	`THUDM/CogVideoX-5b(-I2V)`	Video replay ablations

Table 20: Details of foundation models and generative backends assessed in this study. "Snapshot" records the evaluated API/checkpoint snapshot when no stable public release date is available.

Prompts. We use one prompt template per task type, kept fixed across all models. Representative property-level prompt templates are reproduced in Figs. 33–37.

B.3 Generative Models as Topological World Models

Setup. We evaluate generative models as auxiliary world models rather than as direct answerers. At each step, a reasoner first writes an image or video prompt describing a plausible topological rollout from the current observation. The generator then produces an imagined future observation, and the same reasoner commits to an answer or action from the original observation plus the generated artifact. Image experiments use edit-mode generation anchored to the current PNG observation, with `gpt-image-2`, BAGEL, or ThinkMorph as the image backend. Video experiments use image-to-video rollouts with Wan2.2-I2V-A14B or CogVideoX-style replay backends. Controls keep the reasoner, initial observation, task prompt, and action budget fixed across no-imagination and imagination variants.

Imagined-rollout protocol. For continuity tasks, the imagined artifact is asked to show a path opening, pipe flow, or maze traversal after the candidate manipulation. For separation, the rollout visualizes whether a stroke or object remains one connected piece. For order, it shows the post-fold or post-swap ordering. For enclosure, it shows escape paths, boundary closure, or hole visibility after a viewpoint or state change. For knots, it asks the generator to simulate an endpoint move or strand deformation while preserving over/under crossings. We cap image generation at ten calls per episode and use one video per episode in the end-to-end video setting.

(Generator, Reasoner)	Cont.	Sep.	Ord.	Enc.	Knot.
(`gpt-image-2`, GPT-5.4 mini)	–	$+5.0$	–	–	–
(`gpt-image-2`, InternVL-3.5)	n/a	n/a	n/a	n/a	n/a
(BAGEL, BAGEL reasoner)	n/a	n/a	n/a	n/a	n/a
(ThinkMorph, ThinkMorph reasoner)	n/a	n/a	n/a	n/a	n/a
(Wan2.2-I2V-A14B, MLLM reasoner)	n/a	n/a	n/a	n/a	n/a

Table 21: Generative-imagined evaluation: gain over no-imagination baseline for each (generator, reasoner) pair on the reasoning/planning split. Completed pilot result shown for One-Stroke separation: GPT-5.4 mini improves from 0.0% without imagined images to 5.0% with `gpt-image-2` imagined rollouts over 60 episodes. "n/a" marks configured variants for which no completed aggregate gain is available in the current snapshot.

Failure modes. Imagined rollouts often look locally plausible while violating the exact topological invariant that the task tests. The most common failures are object identity drift across frames, strands or boundaries silently passing through one another, hallucinated openings in closed barriers, and visually smooth but topologically impossible fold or knot transitions. These errors explain why generative rollouts are treated as an auxiliary diagnostic rather than a replacement for the oracle labels.

B.4 Sensitivity to Camera and Viewpoint

Field of view. The benchmark deliberately separates topological difficulty from viewpoint difficulty. Static generators store the scene seed and render camera metadata, so we can rerender the same underlying topology under a canonical top view, oblique view, wider field of view, tighter crop, or distractor-heavy view. For planning environments, the browser viewport is fixed and scene-only screenshots are captured at each step, which prevents accidental resolution changes from becoming a hidden model-specific advantage.

B.5 Do Foundation Models Have Topological Biases?

Connectivity-by-default bias. Models frequently infer a connection from visual proximity. This appears in 2D/3D Maze errors where adjacent rooms, bars, or cells are treated as connected despite a blocking wall, and in planning runs where illegal transitions attempt to move through occupied, blocked, or out-of-bounds cells. In the error analysis for Gemini 3.1 Pro and InternVL, continuity failures are dominated by misclassified connected components rather than by parser failures.

Hole undercounting. Hole Detection exposes a consistent tendency to undercount through-holes, especially in hard scenes with occlusion, multiple cavities, or ambiguous shading. The error taxonomy assigns wrong hole counts to topological invariance errors because the visible object identity is usually correct while the model misclassifies whether a depression, tunnel, or opening contributes a true hole.

Knot chirality bias. Knot and link tasks show a bias toward visually common, low-complexity interpretations. Models often collapse mirror configurations or over/under crossing order into the same answer, which produces errors on link topology, link property, and component-count questions. In the planning version, models also propose endpoint moves that would require strands to pass through one another, indicating that the bias is not merely a static-recognition issue.

C Error Analysis

C.1 Annotation Framework

Motivation. MINDTOPO evaluates whether foundation models can ground visual scenes, identify topological relations, predict state changes under manipulation, and plan valid actions. A single perception-vs.-reasoning split is therefore too coarse: an incorrect answer may come from a bad output format, a mistaken task objective, a visual grounding failure, a wrong belief about topological invariance, an incorrect final-state prediction, an impossible transition, or a poor action plan. We use the following seven primary categories to separate these failure sources.

Top-level categories.

0. Instruction Following Error. The response violates the requested answer protocol or does not provide an interpretable task answer. We assign this category after applying the benchmark's fallback parser, and include cases where the model exhausts its output budget before committing an answer or where the chain-of-thought converges to one value while the final structured field reports a different one.

1. Task Understanding Error. The response is syntactically usable, but the model solves the wrong task, applies the wrong rule, or misunderstands what the prompt is asking. In multi-turn interactive settings this category also covers cases where the first turn frames the task correctly but later turns drift to a different framing of the puzzle.

2. Perception Grounding Error. The model misreads visual evidence, such as object identity, color, location, or the relative position of elements.

3. Topological Invariance Understanding Error. The model misjudges whether a topological relation is preserved or changed under allowed transformations.

4. Feature State Prediction Error. Given a specified action or manipulation, the model predicts the wrong post-action state of a relevant feature.

5. Dynamic Error. The model assumes an impossible transition during motion, such as moving through a wall, obstacle, or topological barrier.

6. Action Planning Error. The model understands the task and scene well enough to act, but selects a poor legal action or multi-step plan.

C.2 Error Category Definitions

C.2.1 0. Instruction Following Error

The model fails to provide a response that can be evaluated under the prompt's requested protocol. This category captures answer-format and response-compliance failures rather than visual or topological capability failures. We assign category 0 after applying the benchmark's fallback parser; if the fallback parser can recover a valid answer, the prediction is analyzed under categories 1–6 instead.

0a. Invalid Answer Format. The output cannot be parsed into the required type or schema. *Examples:* the prompt asks for `{"answer": 3}` but the model returns prose; the JSON key is missing; a cell index is returned as a sentence; the answer type is a list when a scalar is required.

0b. Multiple, Hedged, or Missing Final Answer. The task expects one answer but the model gives several candidates, hedges between options, or never states a final answer. *Examples:* returning "A or C" for a multiple-choice task; listing two candidate moves; explaining the image without an answer.

0c. Prompt-Protocol Violation. The model ignores explicit output constraints even though it appears to understand the broad task. *Examples:* emitting chain-of-thought when only the final answer is requested; using a free-form action when the prompt requires a fixed enum; adding extra fields that break the evaluator.

0d. CoT/Output Internal Inconsistency. The chain-of-thought reaches one conclusion but the structured answer field reports a different value, so the parser commits a verdict that contradicts the model's own stated reasoning. This is distinct from 0a/0b/0c because the response is well-formed

and a single answer is emitted; the failure is purely in the link between rationale and final field. *Examples:* a fence-repair trace concluding "there are zero gaps; done" followed by an answer field of 1; a hole-count walk-through that enumerates three holes followed by an answer field of 4; a knot-classification thought block stating "this is an unknot" followed by the knot label B.

0e. Output-Budget Truncation. The model devotes its output budget to deliberation and is cut off before any structured answer is produced. The fallback parser cannot recover an answer because no structured field was ever emitted. This differs from 0c (intentional protocol violation): the model is trying to comply but never reaches the answer. *Examples:* a long endpoint enumeration on a knot-untangling step that hits the response cap mid-sentence; a multi-hundred-word deliberation on a separation question that ends without a final letter.

C.2.2 1. Task Understanding Error

The response is syntactically usable, but the model has misunderstood the task objective, the permitted operation, or the relevant rule set. This category is different from category 0: the answer may be parseable, but it is generated for the wrong problem.

1a. Wrong Objective. The model optimizes or answers the wrong target. *Examples:* counting objects when the task asks whether an object is enclosed; reporting the current state when the prompt asks for the post-action state; selecting the visually closest option when the task requires topological equivalence.

1b. Wrong Rule Semantics. The model applies an incorrect rule for the environment. *Examples:* treating diagonal grid contact as connectivity when only four-neighbor connectivity is allowed; assuming a pipe piece can connect through a closed side; misunderstanding whether a rope can be lifted over another rope in the specified setting.

1c. Wrong Entity or Reference Binding. The model answers for a different referenced object or option than the one requested by the prompt. *Examples:* comparing option B to the target when the prompt asks about option C; moving the red rope when the instruction refers to the blue rope; using the goal state as the current state in a swap puzzle.

1d. Multi-Turn Task Drift. Specific to interactive tasks. The model identifies the task correctly on the first turn, but in subsequent turns its reasoning context drifts to a different problem framing, causing all later decisions to optimize the wrong objective. Distinct from 1a/1b because the failure is not a one-shot misreading of the prompt; it is a loss of task identity across turns despite the original instructions still being supplied. *Examples:* a one-stroke planner recognizing the rules at step 0 then describing the same board as a robotic-arm pick-and-place scene from step 1 onward; a cat-and-mouse encloser switching to hypothesizing prime-number patterns in the cell labels by the second move; a pipe puzzle reframed as "Unblock Me" or as a sliding-block puzzle once intermediate states are shown.

C.2.3 2. Perception Grounding Error

The model fails at the visual-grounding stage. The relevant task is understood, but the response relies on a wrong visual fact.

2a. Object, Color, or Attribute Misidentification. A visible element is detected but its identity or attribute is wrong. *Examples:* confusing the blue and red ropes; misreading a disk hole as filled; treating a blocked cell as free; mistaking an open pipe side for a closed side.

2b. Location or Relative-Position Grounding Error. The model misreads where elements are or how they are positioned relative to each other. *Examples:* reversing the relative position of A and B; assigning an object to the wrong grid cell; missing that one endpoint lies inside a loop; confusing left/right or above/below relations in the rendered scene.

2c. Missed or Hallucinated Visual Element. A relevant element is absent from or added to the model's internal scene representation, including fine-grained material or depth cues that distinguish a

real opening from a surface marking. *Examples:* missing a hole, sheep, rope crossing, wall segment, or pipe piece; hallucinating an extra obstacle or connection; mistaking a printed symbol or shaded depression for a through-hole, or misreading interior shading as a solid floor.

2d. Multi-Image Correspondence Error. The model parses individual images but fails to align corresponding objects, views, or states. *Examples:* mismatching the current and goal states in a swap puzzle; aligning the wrong point across 3D maze views; confusing a target object with a candidate option in separation tasks.

C.2.4 3. Topological Invariance Understanding Error

The model grounds the scene, but misjudges whether the relevant topological relation is preserved or changed. This category targets topological concepts such as connectivity, enclosure, separation, overlap, crossing, linking, and inside/outside relations under transformations. A common failure is to rely on visible coordinates or shape similarity while missing that topology has changed, or to infer a topological change from a purely geometric deformation.

3a. False Invariance. The model says a topological relation is unchanged when it has changed. *Examples:* concluding that two configurations are equivalent because endpoints or object positions look unchanged, even though a rope crossing, overlap, enclosure, or connectivity relation has changed.

3b. False Change. The model says a topological relation changes under a transformation that preserves it. *Examples:* treating translation, rotation, stretching, or a viewpoint change as breaking connectivity, enclosure, or linkage when the topological relation remains the same.

3c. Wrong Topological Relation. The model reasons about the correct objects but assigns the wrong topological relation. *Examples:* linked vs. unlinked ropes; connected vs. separated pipe components; inside vs. outside a loop; trapped vs. reachable in a maze-like enclosure.

C.2.5 4. Feature State Prediction Error

The model understands the instruction and initial scene, but predicts the wrong final state of a feature after a specified manipulation. This category concerns the resulting state, not whether the intermediate motion was physically possible. For example, after the prompt describes lifting a blue rope and placing it in the upper-left region, the model may predict that the blue rope no longer overlaps the red rope, even though the ground-truth final state still contains an overlap.

4a. Wrong Post-Action Topological Feature. The predicted final connectivity, overlap, crossing, enclosure, or separation state is wrong. *Examples:* predicting that a moved rope no longer overlaps another rope when it still does; predicting that a rotated pipe network is connected when one joint remains disconnected; predicting that a blocked cell traps the cat when an escape route remains.

4b. Wrong Post-Action Visual or Discrete State. The model applies the action to the wrong state variable. *Examples:* wrong post-rotation orientation of a pipe or laser disk; wrong final slot after a swap; wrong cell status after placing a block; wrong endpoint location after a move.

4c. Incomplete State Update. The model updates one feature but leaves another dependent feature in the old state. *Examples:* moving a rope endpoint but not updating crossings; rotating a disk but not updating the number of transmitted holes; blocking a cell but not updating reachability.

C.2.6 5. Dynamic Error

The model assumes an impossible intermediate transition during motion. Unlike category 4, which concerns the final state after an action, category 5 concerns whether the path from the initial state to the final state violates the environment's dynamics or physical constraints.

5a. Barrier or Collision Violation. The model moves through an obstacle, wall, boundary, or occupied cell. *Examples:* planning a maze step through a wall; sliding a block through another block; routing a path through a forbidden cell; moving the cat through a blocked cell.

5b. Topological Barrier Violation. The model permits a transition that would require crossing, cutting, or teleporting through a topological constraint that the task forbids. *Examples:* passing a rope through another rope without an allowed lift; untangling a knot by crossing strands through each other; separating linked components through an impossible motion.

5c. Unsupported Continuous Motion. The model predicts a valid-looking final arrangement but provides or assumes no legal continuous path to reach it. *Examples:* moving an object from one enclosed region to another without crossing a boundary; rotating a component through a collision; placing a feature behind an occluding barrier as if it could pass through it.

C.2.7 6. Action Planning Error

Specific to interactive tasks. The model follows the output protocol, understands the task, grounds the relevant scene facts, and does not assume an impossible transition, but still chooses a poor legal action or action sequence.

6a. Legal but Irrelevant Action. The action is allowed but does not advance the objective. *Examples:* rotating a non-critical pipe piece; blocking a cell that is not on any escape route; moving a puzzle tile that leaves the state equally far from the goal; adjusting a rope segment that does not reduce crossings.

6b. Short-Horizon or Trap-Inducing Plan. The action appears locally reasonable but causes later failure. *Examples:* blocking a cell that lets the cat escape elsewhere; forming a one-stroke path that cuts off an unvisited region; making a swap that increases the minimum remaining distance; untangling one crossing while creating a worse downstream crossing.

6c. State-Tracking Planning Error. The model chooses a later action based on a stale but otherwise valid state estimate. *Examples:* forgetting a previously rotated pipe piece; treating an already blocked cell as free; planning from the initial rather than current puzzle state; repeating an action that has already been applied.

C.3 Cross-Model and Cross-Task Distributions

Sampling protocol. For each (model, task) pair we collected all incorrect predictions, drew up to 35 errors uniformly at random (all errors when fewer than 35 were available), assigned each sampled prediction one primary category, and projected the per-sample distribution onto the absolute population by $\widehat{n}_c = (n_c^{\text{sample}}/n^{\text{sample}}) \cdot N$, where N is the total number of errors for that pair. The perception split contains 8 static-perception tasks (continuity-2d-maze, continuity-3d-maze, enclosure-hole-detection, enclosure-sheep, knots-static, order-bead-string, order-origami, separation-objects). The planning split contains 5 interactive tasks (continuity-pipe, enclosure-chat-noir, knots-untangle, order-swap-puzzle, separation-one-stroke). Figures 40 and 41 provide the environment-level and model-level breakdowns.

Per-property breakdown. We report category frequencies separately for each topological property so that perception failures, invariance failures, state-prediction failures, dynamic violations, and planning failures are not collapsed into one aggregate error rate.

Comparison with human annotators. The same category definitions can be applied to human responses. This lets us distinguish human-like task or perception mistakes from model- specific failures such as invalid output formatting, brittle topological invariance judgments, or impossible dynamic transitions.

D Extended Related Work Discussion

This appendix expands the condensed Related Work in Section 4 into the longer, per-work discussion that did not fit in the main body.

Topological Cognition. Cognitive development positions topology as a primitive layer of spatial cognition that precedes projective and Euclidean reasoning. Piaget and Inhelder [1] introduced

Category	Gemini-3.1-Pro count	%	InternVL3.5-241B count	%	Combined count	%
0. Instruction following	211	6.6	1095	20.3	1306	15.2
1. Task understanding	98	3.1	0	0.0	98	1.1
2. Perception grounding	1564	48.7	2945	54.7	4509	52.5
3. Topological invariance	1140	35.6	1174	21.8	2314	26.9
4. Feature state prediction	196	6.1	172	3.2	368	4.3
5. Dynamic violation	0	0.0	0	0.0	0	0.0
6. Action planning	0	0.0	0	0.0	0	0.0
Total errors	3205	100	5389	100	8594	100

Table 22: Reasoning split, task-weighted projected error counts and percentages.

Category	Gemini-3.1-Pro count	%	InternVL3.5-241B count	%	Combined count	%
0. Instruction following	0	0.0	201	21.4	201	16.7
1. Task understanding	0	0.0	553	58.8	553	45.8
2. Perception grounding	4	1.5	19	2.0	23	1.9
3. Topological invariance	0	0.0	0	0.0	0	0.0
4. Feature state prediction	33	12.4	0	0.0	33	2.7
5. Dynamic violation	86	32.1	32	3.4	117	9.7
6. Action planning	144	54.0	136	14.4	280	23.2
Total errors	267	100	941	100	1208	100

Table 23: Planning split, projected error counts and percentages.

the original five-class taxonomy—proximity, separation, order, enclosure, and continuity—with subsequent mathematical and constructionist extensions by Beth and Piaget [27] and Papert [28]. Adult perception inherits the same primacy: Chen [2, 3] showed that the visual system extracts topological invariants such as connectedness and number of holes ahead of feature-based attributes. Holes have been formalized as dependent entities by Casati and Varzi [29], with perceptual evidence on surrounded regions and hole shape from Nelson and Palmer [30] and Bertamini and Croucher [31]. Knots constitute a distinct cognitive domain—characterized by Strohecker [9] as the "mother structure" that coordinates the other relations, and shown by Croom and Firestone [13] to strain adult intuitive physics. Hatcher [11] provides the algebraic-topology formalization, Martin [10] clarifies how Piagetian tasks map onto formal topological constructs, and Gärdenfors [32] argues that natural categories rest on topological notions such as connectedness and convexity. MINDTOPO translates this body of evidence into a unified visual taxonomy and asks whether modern foundation models exhibit the same topological primacy.

Cognitively Grounded and Topology-Adjacent Benchmarks. A closer line of work either revisits developmental psychology or isolates a single topological task, but no prior benchmark unifies a topology-flavored taxonomy for vision-grounded MLLM evaluation. Piaget-inspired benchmarks diagnose developmental gaps in foundation models: CogDevelop2K [33] and CogLM [34] stage tests of reversed cognitive development, ConserveBench [35] and PerspectBench [36] target conservation and perspective-taking, and KiVA [37] and BabyVision [38] extend the developmental lens to visual analogy and pre-linguistic vision; each, however, evaluates abilities orthogonal to topological invariants. A complementary thread targets individual topology-adjacent tasks in isolation: AlphaMaze [39] for path connectivity, KnotGym [8] for knots, ORIGAMISPACE [40] and GamiBench [41] for folding, OPTiCAL [42] for abstract positional reasoning, and Thinking in Structures [43] for constrained-manifold and symbolic puzzles. Closest in spirit, Wang et al. [44] probe topological versus geometric concept sensitivity in vision models. Each of these works, however, covers only a single property; MINDTOPO unifies five topological properties under both reasoning and planning into a single, cognitively grounded suite.

Spatial Reasoning Benchmarks for MLLMs. Most spatial-reasoning benchmarks for multimodal large language models target Euclidean and viewpoint-centric competence rather than topological invariance. Static VQA benchmarks measure metric properties such as distance, direction, and inter-object relations: SpatialVLM [4] and SpatialMQA [45] ground this line, while broader aggregations including OmniSpatial [7], SITE [46], 3DSRBench [47], SPATIAL-DISE [48], and Mind the Gap [49] expand coverage but stay within a metric grammar. Cognitive-map and viewpoint-integration benchmarks such as Thinking in Space [5], MindCube [6], and Theory of Space [50] probe spatial memory and limited-view inference, and interactive evaluations including iVISPAR [51], VSP [52], and RoboSpatial [53] test multi-step spatial control and affordance use. None of these isolate whether a model preserves invariants under continuous deformation: the questions all hinge on quantitative geometry or viewpoint, not on qualitative spatial structure. MINDTOPO complements this line by testing topology-flavored competence that is invariant to metric distortion.

World Models and Physical Reasoning. A separate line of work evaluates MLLMs and video models as world simulators or physical reasoners, with focus on physical fidelity rather than topology preservation. Generative world-model benchmarks such as PhyGenBench [55], Physion-Eval [56], EWMBench [57], WorldArena [58], World-in-World [59], and PhysicsMind [60] score image and video rollouts on commonsense and dynamics realism. Reasoning-oriented evaluations including MMGR [61], TiViBench [62], and VideoThinkBench [63] extend this view to multi-modal generative reasoning, and ENACT [64] casts embodied cognition as forward and inverse world modeling from egocentric interaction. Adjacent agentic benchmarks such as PhysBench [65], DeepPHY [66], and CHAIN [67] emphasize physics- and affordance-level reasoning. Topology-relevant phenomena—objects passing through holes, surfaces being cut or sealed, ropes tightening or untangling—either appear only incidentally or are scored by metric and visual fidelity rather than by invariance under continuous deformation. MINDTOPO complements this line with topology-grounded probes that decouple topology preservation from physical realism, providing a controlled diagnostic for whether MLLMs internalize the qualitative structure of the physical world.

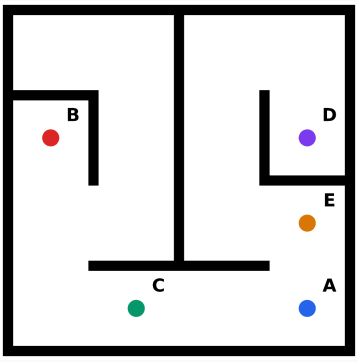
E The Use of Large Language Models

We used large language models (LLMs), including Google's Gemini 3.1 Pro and OpenAI's GPT 5.5, as auxiliary tools to assist with writing, editing, and conducting the literature review for this manuscript. All content was critically revised and fact-checked by the human authors to ensure its scientific validity and originality. The authors are fully responsible for all statements and conclusions presented in this paper. Specifically, we used LLMs for polishing wording and writing, and to retrieve related works.

Reasoning Task: Continuity 2D Maze (Reachability Set)

[Task]
You are looking at a top-down view of a 2D maze. Several cells are marked with labeled circles (A, B, C, ...). Your job is to decide, for the target point, which of the OTHER labeled points you can reach from it through the open maze corridors. A point Y is reachable from X exactly when you can travel from X to Y without crossing any wall. You can only step from one cell to a neighboring cell across an open edge — you cannot cut diagonally through a wall corner.
Wall types: Walls in the maze come in three visual styles, all of which block movement equally: (a) full edge walls — solid lines along an entire cell edge; (b) partial edge walls — solid line segments shorter than a full edge; (c) diagonal walls — solid lines cutting across a cell along its diagonal.
Any solid line, regardless of length or orientation, is a wall and cannot be crossed. Each labeled circle marks one cell.
The visual evidence for this question is provided below.

[Image]



[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.
3. Do not output explanation beyond the required final answer.

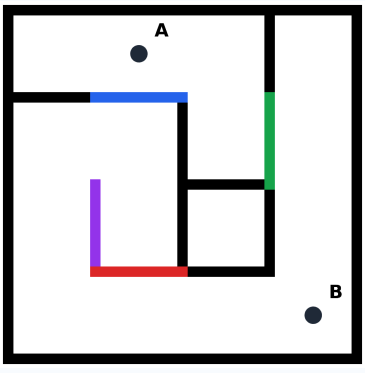
[Question]
Which of the other labeled points can you reach from point A?
The other labeled points in this maze are: B, C, D, E.

[Answer Format]
Output exactly one JSON object: {\"answer\": [\"B\", \"D\"]} and nothing else.
Replace [\"B\", \"D\"] with the actual answer: a JSON array of point names, picked from the other labeled points shown in the image and NOT including the target itself, listed in alphabetical order. Use [] if no other point is reachable.

Reasoning Task: Continuity 2D Maze (Bar Removal)

[Task]
You are looking at a top-down view of a 2D maze. Two cells are marked with labeled circles A and B; in the current maze they are blocked from each other. Some of the maze's internal walls have been recolored as colored bars (purple / red / green / blue / yellow / orange). Suppose you are allowed to remove EXACTLY ONE bar — which colors of bar, when removed alone, would let A and B reach each other? Evaluate each bar independently: imagine removing only that one bar, leave every other bar in place, and check whether a path from A to B opens up. List EVERY color that works.
Wall types: Walls in the maze come in three visual styles, all of which block movement equally: (a) full edge walls — solid lines along an entire cell edge; (b) partial edge walls — solid line segments shorter than a full edge; (c) diagonal walls — solid lines cutting across a cell along its diagonal. Any solid line, regardless of length or orientation, is a wall and cannot be crossed. Each labeled circle marks one cell. A bar is a colored (non-black) wall; black walls are NOT bars.
Scoring: your answer is correct ONLY if you list every color whose single-bar removal reconnects A and B — no missing colors and no extras. A partial list, an extra color, or an empty answer when at least one qualifying bar exists, all count as wrong.
The visual evidence for this question is provided below.

[Image]



[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.
3. Do not output explanation beyond the required final answer.

[Question]
Identify EVERY color of bar that, if removed alone, reconnects point A and point B (which are currently blocked from each other). List all qualifying colors — missing any one of them counts as wrong. The bars in this maze are colored: blue, green, purple, red.

[Answer Format]
Output exactly one JSON object: {\"answer\": [\"green\", \"red\"]} and nothing else.\nReplace [\"green\", \"red\"] with the actual answer: a JSON array of color names from the bars shown in the image, in alphabetical order. Use [] only if NO single-bar removal connects them.

Figure 8: **2D Maze** qualitative examples across the three difficulty tiers.

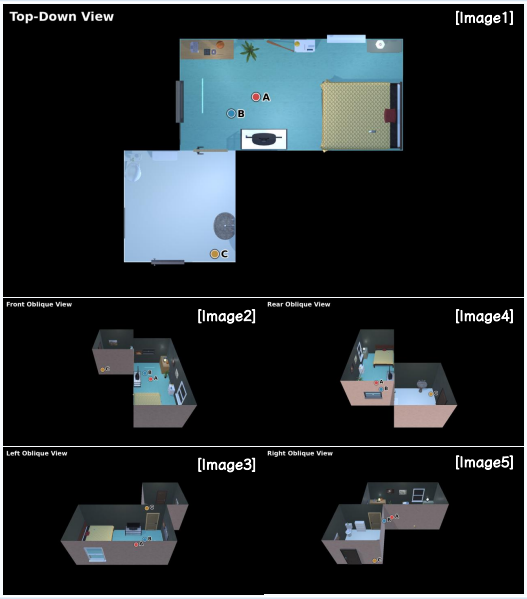
Reasoning Task: Continuity 3D Maze (Point List)

[Task]
You are solving a topological task called 3D Maze under the continuity category.
In this task, you must list every other labeled point that is connected to the target point under the shown door states.
If this task depends on a specific visual definition, use this definition exactly: two labeled points are connected only if an agent can travel between them without crossing walls, closed doors, furniture, or other solid obstacles. Open doors and open floor/corridor spaces are passable. Closed doors are not passable.
The visual evidence for this question is provided below.

[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.
3. Do not output explanation beyond the required final answer.

[Question]
List all other labeled points that are connected to point C under the shown door states. The other labeled points are: A, B.

[Answer Format]
Output exactly one JSON object: {\"answer\":{ans}} and nothing else.\nReplace {ans} with the actual answer: a JSON array of point names, excluding the target point itself, listed in alphabetical order. Use [] if no other labeled point is connected to the target.



Reasoning Task: Continuity 3D Maze (Subset Choice)

[Task]
You are solving a topological task called 3D Maze under the continuity category.
In this task, you must determine which candidate option lists labeled points that are all mutually connected under the shown door states.
If this task depends on a specific visual definition, use this definition exactly: two labeled points are connected only if an agent can travel between them without crossing walls, closed doors, furniture, or other solid obstacles. Open doors and open floor/corridor spaces are passable. Closed doors are not passable.
The visual evidence for this question is provided below

[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.
3. Do not output explanation beyond the required final answer.

[Question]
Which option lists labeled points that are all mutually connected under the shown door states?
Each option contains two or more labeled points.
A listed option is correct only if every point in that option can reach every other point in the same option through a collision-free navigable path.
Exactly one option is correct. Choose one option from 1, 2, 3, 4.

Option 1: [A, C]
Option 2: [B, C]
Option 3: [A, B]
Option 4: [A, B, C]

[Answer Format]
Output exactly one JSON object: {\"answer\":\"{ans}\"} and nothing else.
Replace {ans} with the single legal answer for this task, chosen from \"1\", \"2\", \"3\", \"4\".

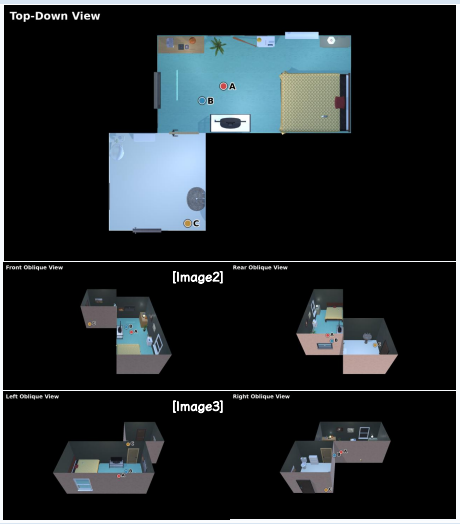
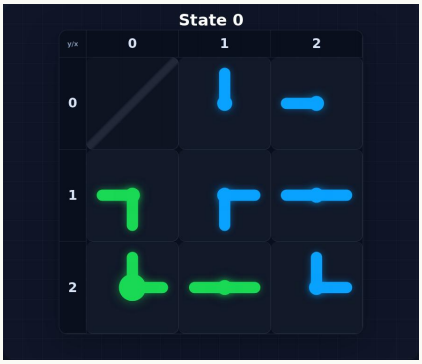


Figure 9: **3D Maze** qualitative examples.

Interactive Task: Continuity Pipe

[Task]
You are solving Continuity Pipe.
In this task, you must rotate pipe pieces 90 degrees clockwise per turn until every pipe connects back to the green source. The board is a square grid. Some cells contain rotatable pipe pieces and some cells may be empty.
The green source pipe is the starting source. Pipes connected to the source are green; pipes not connected to the source are blue.
Your goal is to rotate every non-empty pipe cell until every pipe on the board is connected to the green source.
The x coordinates are shown above the grid and the y coordinates are shown on the left side of the grid.

The board size is 3x3.
Legal actions for this turn:
(x=1, y=0), (x=2, y=0), (x=0, y=1), (x=1, y=1), (x=2, y=1), (x=0, y=2), (x=1, y=2), (x=2, y=2).
To solve this task, output the next legal action for the current state.



[Rules]
1. At each turn, choose exactly one non-empty pipe cell.
2. The selected pipe rotates clockwise by 90 degrees.
3. A legal action must be one of the listed (x, y) coordinates for the current state.
4. Empty cells are not legal actions.
5. The task succeeds when every pipe is connected to the source.
6. At each turn, output exactly one next action for the current state.

[Answer Format]
Output exactly one JSON object: {"answer":{"x": {x}, "y": {y}}} and nothing else.
Replace {x} and {y} with the selected legal grid coordinate, wrapped inside the "answer" field.

[Current Task]
The current state image is shown below.

Figure 10: **Pipe Rotation** qualitative examples.

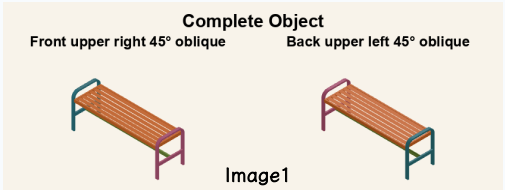
Reasoning Task: Sepraration Objects (Bench Sjalland)

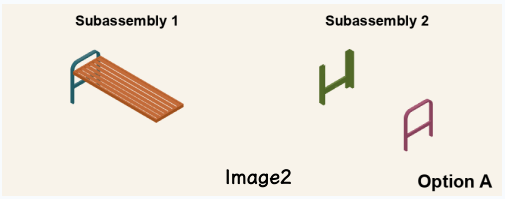
[Task]
You are solving a topological task called Separation Objects under the separation category.
In this task, you must determine which candidate option shows the correct two-part decomposition of the complete object. The correct option must split the complete object into exactly two candidate subassemblies that together form the complete object, with no missing parts, no extra parts, and no parts from another object.
Image 1 contains two labeled views of the complete object: Front upper right 45-degree oblique and Back upper left 45-degree oblique. Each option image contains the two candidate subassemblies for one option; each subassembly is shown from the front upper right 45-degree oblique view.
The visual evidence for this question is provided below.

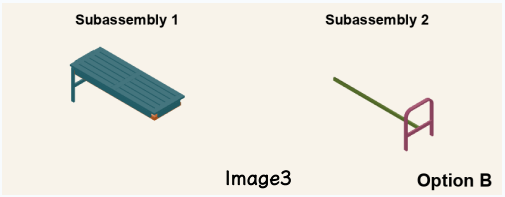
[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.
3. Do not output explanation beyond the required final answer.

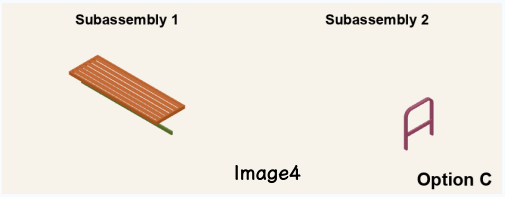
[Question]
Which option shows the correct two-part decomposition of the complete object?
Choose one option from A, B, C, D, E.

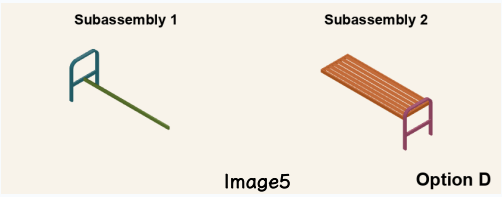
[Answer Format]
Output exactly one JSON object:
{\"answer\":\"{ans}\"} and nothing else.
Replace {ans} with the single legal answer for this task, chosen from \"A\", \"B\", \"C\", \"D\", \"E\".











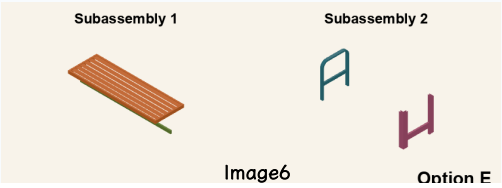


Figure 11: **IKEA** qualitative examples.

Reasoning Task: Sepraration Objects (Bench Applaro)

[Task]
You are solving a topological task called Separation Objects under the separation category.
In this task, you must determine which candidate option shows the correct two-part decomposition of the complete object. The correct option must split the complete object into exactly two candidate subassemblies that together form the complete object, with no missing parts, no extra parts, and no parts from another object.
Image 1 contains two labeled views of the complete object: Front upper right 45-degree oblique and Back upper left 45-degree oblique. Each option image contains the two candidate subassemblies for one option; each subassembly is shown from the front upper right 45-degree oblique view.
The visual evidence for this question is provided below.

[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.
3. Do not output explanation beyond the required final answer.

[Question]
Which option shows the correct two-part decomposition of the complete object?
Choose one option from A, B, C, D, E.

[Answer Format]
Output exactly one JSON object:
{\"answer\":\"{ans}\"} and nothing else.
Replace {ans} with the single legal answer for this task, chosen from \"A\", \"B\", \"C\", \"D\", \"E\".

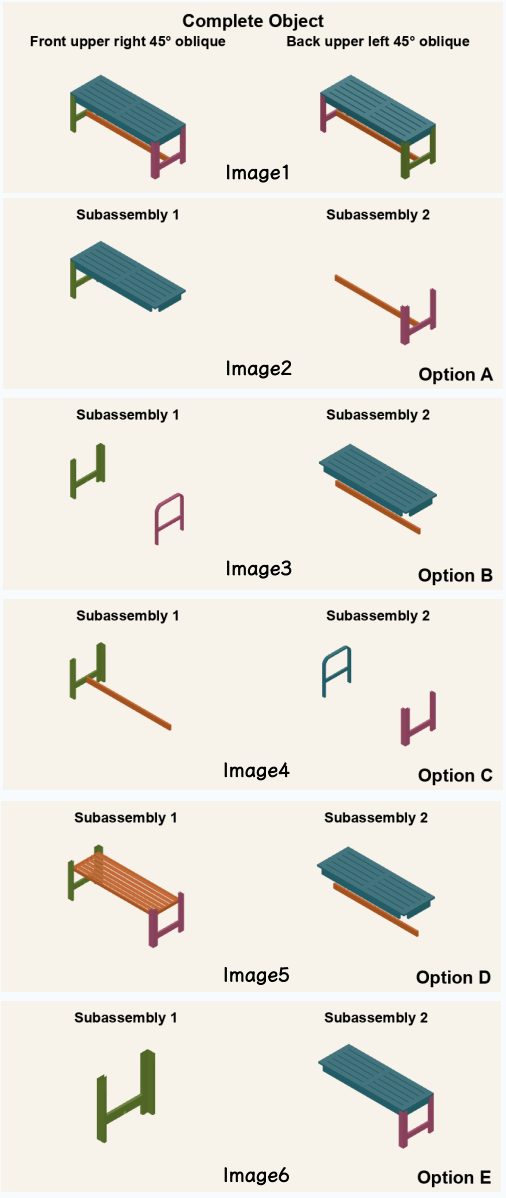


Figure 12: **IKEA** qualitative examples.

Reasoning Task: Sepraration Objects (Chair Agam)

[Task]
You are solving a topological task called Separation Objects under the separation category.
In this task, you must determine which candidate option shows the correct two-part decomposition of the complete object. The correct option must split the complete object into exactly two candidate subassemblies that together form the complete object, with no missing parts, no extra parts, and no parts from another object.
Image 1 contains two labeled views of the complete object: Front upper right 45-degree oblique and Back upper left 45-degree oblique. Each option image contains the two candidate subassemblies for one option; each subassembly is shown from the front upper right 45-degree oblique view.
The visual evidence for this question is provided below.

[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.
3. Do not output explanation beyond the required final answer.

[Question]
Which option shows the correct two-part decomposition of the complete object? Choose one option from A, B, C, D, E.

[Answer Format]
Output exactly one JSON object: {\"answer\":\"{ans}\"} and nothing else.\nReplace {ans} with the single legal answer for this task, chosen from \"A\", \"B\", \"C\", \"D\", \"E\".

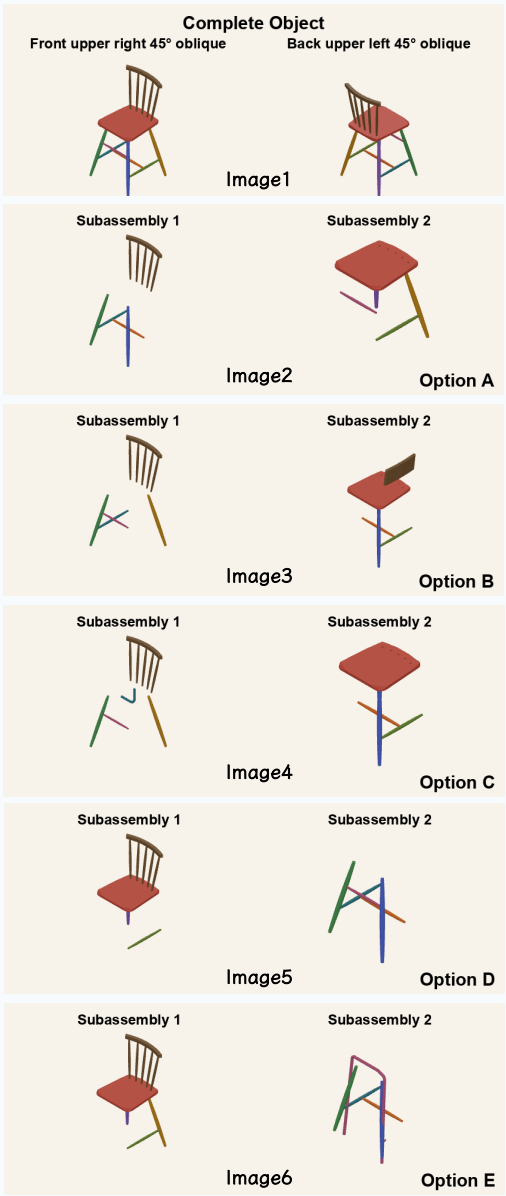


Figure 13: **IKEA** qualitative examples.

Reasoning Task: Sepraration Objects (Chair Applaro)

[Task]
You are solving a topological task called Separation Objects under the separation category.
In this task, you must determine which candidate option shows the correct two-part decomposition of the complete object. The correct option must split the complete object into exactly two candidate subassemblies that together form the complete object, with no missing parts, no extra parts, and no parts from another object.
Image 1 contains two labeled views of the complete object: Front upper right 45-degree oblique and Back upper left 45-degree oblique. Each option image contains the two candidate subassemblies for one option; each subassembly is shown from the front upper right 45-degree oblique view.
The visual evidence for this question is provided below.

[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.
3. Do not output explanation beyond the required final answer.

[Question]
Which option shows the correct two-part decomposition of the complete object? Choose one option from A, B, C, D, E.

[Answer Format]
Output exactly one JSON object: {\"answer\":\"{ans}\"} and nothing else. Replace {ans} with the single legal answer for this task, chosen from \"A\", \"B\", \"C\", \"D\", \"E\".

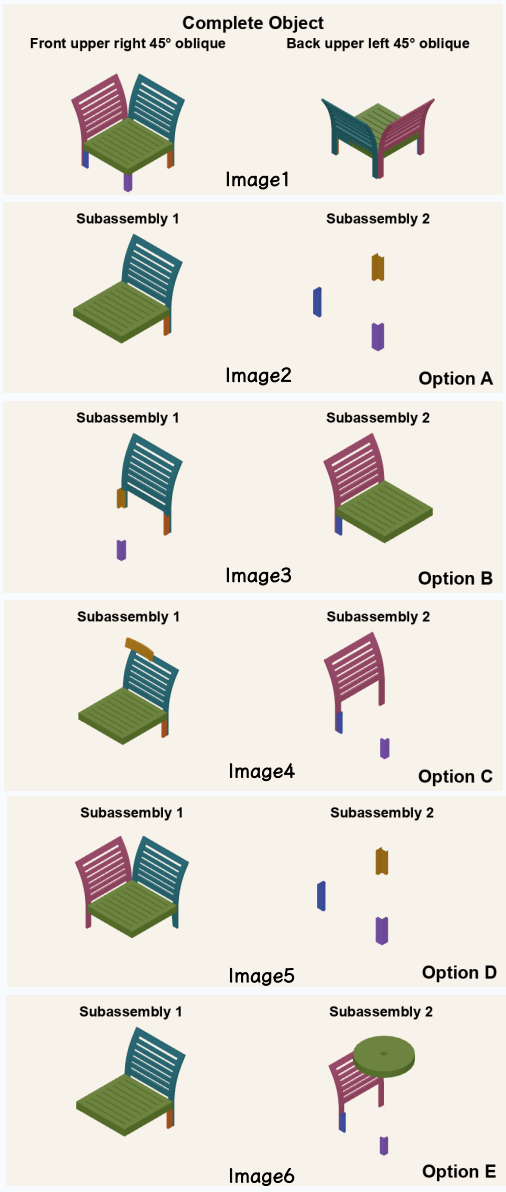


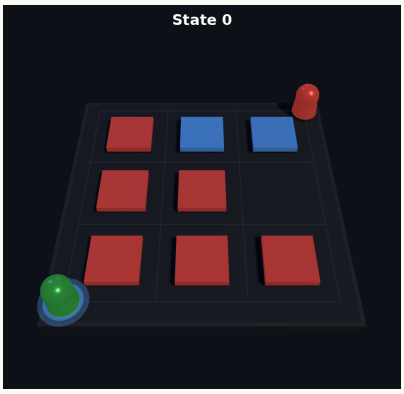
Figure 14: **IKEA** qualitative examples.

Interactive Task: Continuity Pipe

[Task]
You are solving One-Stroke Color Grouping.
In this task, you must draw one continuous stroke from the bottom-left to the top-right so same-colored cells stay together and different colors are separated.
In this task, you must extend the current stroke from the bottom-left start vertex to the top-right goal vertex so that same-colored cells end up in the same region and different-colored cells end up in different regions.
In each board image, the white stroke is the current path, the green peg marks the start, and the red peg marks the goal.

Legal actions for this turn: U, R.
To solve this task, output the next legal action for the current state.

[Image]



[Rules]
1. At each turn, choose exactly one move from U, D, L, or R to extend the current stroke by one edge. U means up, D means down, L means left, and R means right.
2. A legal action must be one of the legal actions listed for the current state. You must not reuse an edge or create a closed loop. Backtracking over the most recent edge is allowed and acts like undoing the last move.
3. After each action, the environment returns the next state. Illegal actions keep the board state unchanged but still count as a step.
4. The task is solved when the stroke reaches the top-right goal vertex and the final regions satisfy the color-separation rule.
5. The episode ends when the puzzle reaches a done state or the step budget is exhausted.
6. At each turn, output exactly one next action for the current state.

[Answer Format]
Output exactly one JSON object: {"answer":"{ans}"} and nothing else.
Replace {ans} with the single legal answer for this task, chosen from "U", "D", "L", "R", using the JSON shape {"answer":"{ans}"}.

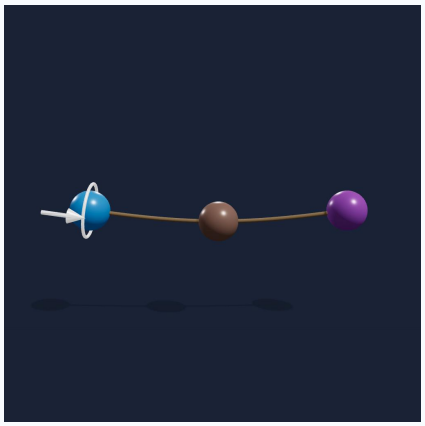
[Current Task]
The current state image is shown below.

Figure 15: **One-Stroke** qualitative examples.

Reasoning Task: Order Bead String (Describe Sequence)

[Task]
You are solving the order task.
In this task, you must trace a bead string continuously from one endpoint to the other and report the bead colors in encounter order.
If this task depends on a specific visual definition, use this definition exactly: A bead string is a rope threaded through colored beads. The bead sequence is determined by following the rope continuously from one visible endpoint to the other and listing bead colors in the exact order encountered along that path.
The visual evidence for this question is provided below.

[Image]



[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.
3. Do not output explanation beyond the required final answer.
4. Bead colors in these images are exactly: RED, BLUE, GREEN, YELLOW, ORANGE, PURPLE, WHITE, BROWN. Use these exact uppercase names; do not invent synonyms (e.g. write 'PURPLE', not 'VIOLET' or 'MAGENTA').

[Question]
Look at this 3D image of a curved string with colored beads threaded on it.
Trace the string from one end to the other and list all bead colors in the order you encounter them.

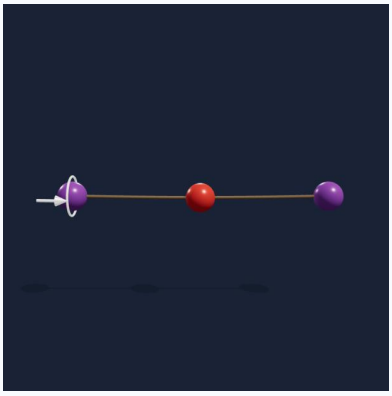
[Answer Format]
Output exactly one JSON object: {\"answer\": {value}} and nothing else.
Replace {value} with the single legal answer for this task, chosen from a single JSON string containing the comma-separated bead-color sequence, for example \"RED, BLUE, GREEN\".

Figure 16: Bead String qualitative examples.

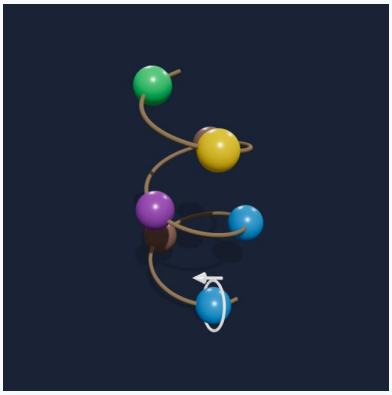
Reasoning Task: Order Bead String (Pair Relationship)

[Task]
You are solving the order task. In this task, you must compare the bead-color sequences shown in two bead-string images and classify their relationship.
If this task depends on a specific visual definition, use this definition exactly: A bead string is a rope threaded through colored beads. IDENTICAL means the two sequences match exactly; REVERSED means one is the exact reverse of the other; CYCLIC_ROTATION means one is a cyclic shift of the other; DIFFERENT means none of the above.
The visual evidence for this question is provided below.

[Image1]



[Image2]



[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.
3. Do not output explanation beyond the required final answer.
4. Bead colors in these images are exactly: RED, BLUE, GREEN, YELLOW, ORANGE, PURPLE, WHITE, BROWN. Use these exact uppercase names; do not invent synonyms (e.g. write 'PURPLE', not 'VIOLET' or 'MAGENTA').

[Question]
You are shown two images of bead strings. Each string has colored beads on a curved rope.
Compare the bead color sequences on both strings and determine their relationship.
Ignore the shape of the string — focus only on the sequence of bead colors.

[Answer Format]
Output exactly one JSON object: {\"answer\": {value}} and nothing else. Replace {value} with the single legal answer for this task, chosen from one of the JSON strings \"IDENTICAL\", \"REVERSED\", \"CYCLIC_ROTATION\", or \"DIFFERENT\".

Figure 17: Bead String qualitative examples.

Reasoning Task: Origami Point Tracking (Bird-base)

[Task]
You are solving origami point tracking.
In this task, you must identify which labeled points on a folded piece of paper correspond to the unmarked dots placed on the original flat (unfolded) paper.
The first image shows the flat paper with 2 unmarked dot(s) printed on it.
The following images show the same paper being folded step by step, ending with the fully folded result where every visible point is labeled using the letters A, B, C, D, E.
Paper is opaque: points on the back side may be hidden by overlying layers, and the same physical dot may appear in different positions across steps as the paper folds.
If the paper rotates during the sequence, it rotates slowly. You must track each unmarked dot through the fold sequence and report the letter(s) in the final image that correspond to the original unmarked dot(s).
If this task depends on a specific visual definition, use this definition exactly: not applicable
The visual evidence for this question is provided below.

[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.
3. Do not output explanation beyond the required final answer.

[Question]
Which lettered point(s) in the final folded image correspond to the unmarked dot(s) shown on the flat paper in the first image? List the matching letter(s).

[Answer Format]
Output exactly one JSON object: {\"answer\": {json_answer_value}} and nothing else.
Replace {json_answer_value} with the single answer for this task, formatted according to a JSON array of uppercase letters chosen from [A, B, C, D, E], with no duplicates, e.g. [\"A\", \"C\"].

[Images]

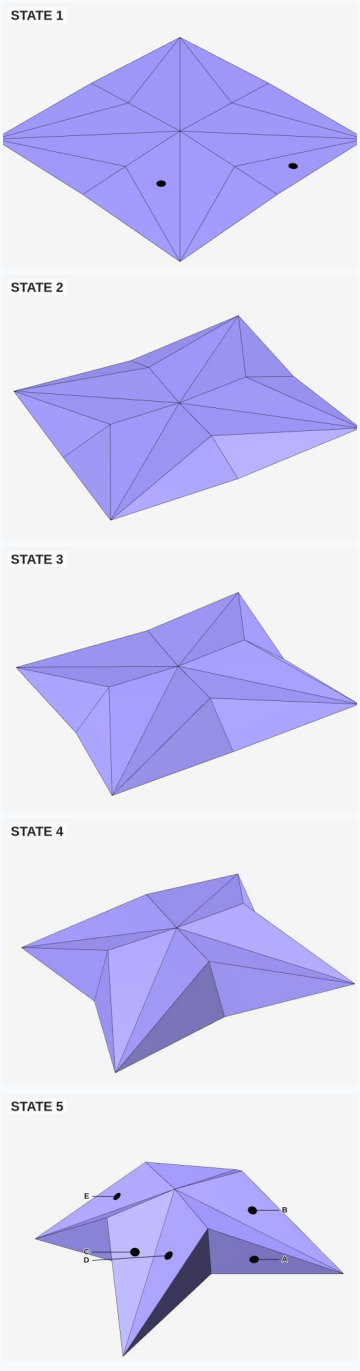


Figure 18: **Origami Point Tracking** qualitative examples across the four bases and three difficulty tiers.

Reasoning Task: Origami Point Tracking (Opensink-based)

[Task]
You are solving origami point tracking.
In this task, you must identify which labeled points on a folded piece of paper correspond to the unmarked dots placed on the original flat (unfolded) paper.
The first image shows the flat paper with 2 unmarked dot(s) printed on it.
The following images show the same paper being folded step by step, ending with the fully folded result where every visible point is labeled using the letters A, B, C, D, E, F.
Paper is opaque: points on the back side may be hidden by overlying layers, and the same physical dot may appear in different positions across steps as the paper folds.
If the paper rotates during the sequence, it rotates slowly.
You must track each unmarked dot through the fold sequence and report the letter(s) in the final image that correspond to the original unmarked dot(s).
If this task depends on a specific visual definition, use this definition exactly: not applicable
The visual evidence for this question is provided below.

[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.
3. Do not output explanation beyond the required final answer.

[Question]
Which lettered point(s) in the final folded image correspond to the unmarked dot(s) shown on the flat paper in the first image? List the matching letter(s).

[Answer Format]
Output exactly one JSON object: {\"answer\": {json_answer_value}} and nothing else.
Replace {json_answer_value} with the single answer for this task, formatted according to a JSON array of uppercase letters chosen from [A, B, C, D, E, F], with no duplicates, e.g. [\"A\", \"C\"].

[Images]

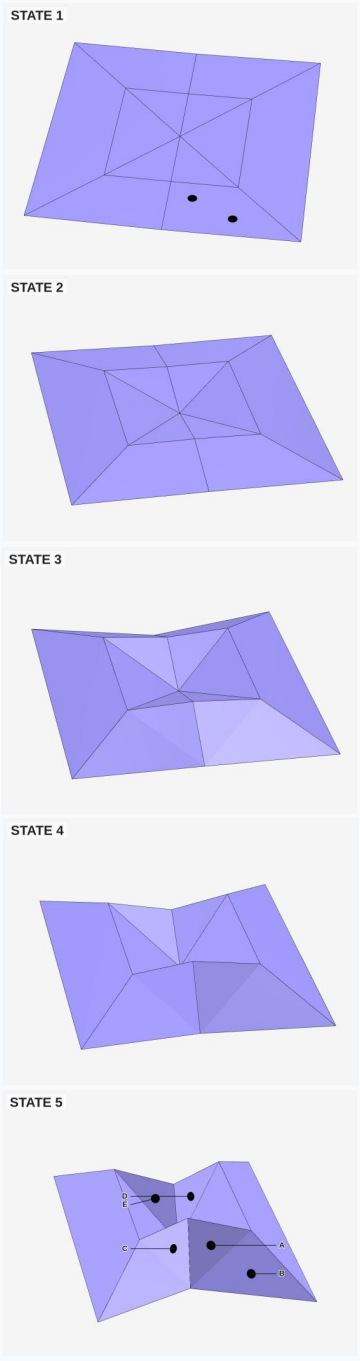


Figure 19: **Origami Point Tracking** qualitative examples across the four bases and three difficulty tiers.

Reasoning Task: Origami Point Tracking (Pinwheel-base)

[Task]
You are solving origami point tracking.
In this task, you must identify which labeled points on a folded piece of paper correspond to the unmarked dots placed on the original flat (unfolded) paper.
The first image shows the flat paper with 2 unmarked dot(s) printed on it.
The following images show the same paper being folded step by step, ending with the fully folded result where every visible point is labeled using the letters A, B, C, D, E.
Paper is opaque: points on the back side may be hidden by overlying layers, and the same physical dot may appear in different positions across steps as the paper folds.
If the paper rotates during the sequence, it rotates slowly.
You must track each unmarked dot through the fold sequence and report the letter(s) in the final image that correspond to the original unmarked dot(s).
If this task depends on a specific visual definition, use this definition exactly: not applicable
The visual evidence for this question is provided below.

[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.
3. Do not output explanation beyond the required final answer.

[Question]
Which lettered point(s) in the final folded image correspond to the unmarked dot(s) shown on the flat paper in the first image? List the matching letter(s).

[Answer Format]
Output exactly one JSON object: {\"answer\": {json_answer_value}} and nothing else.
Replace {json_answer_value} with the single answer for this task, formatted according to a JSON array of uppercase letters chosen from [A, B, C, D, E], with no duplicates, e.g. [\"A\", \"C\"].

[Images]

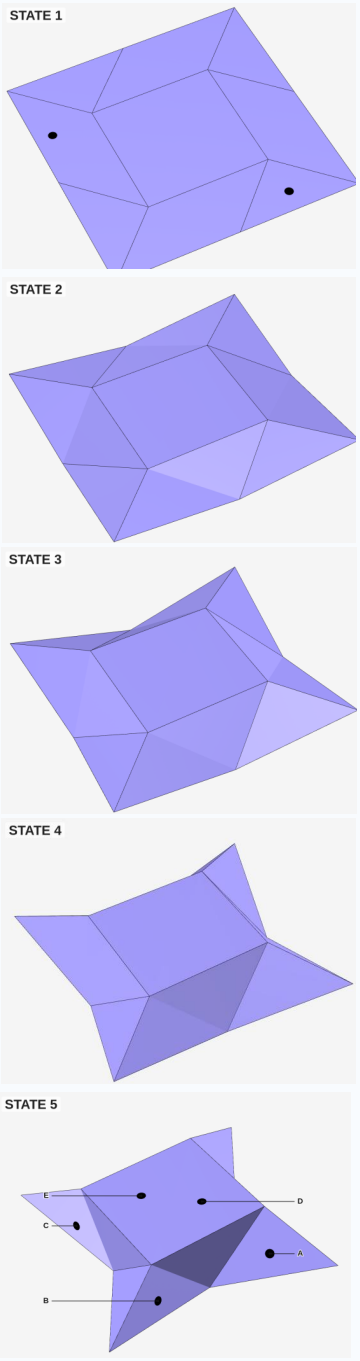


Figure 20: **Origami Point Tracking** qualitative examples across the four bases and three difficulty tiers.

Reasoning Task: Origami Point Tracking (Waterbomb-base)

[Task]
You are solving origami point tracking.
In this task, you must identify which labeled points on a folded piece of paper correspond to the unmarked dots placed on the original flat (unfolded) paper.
The first image shows the flat paper with 2 unmarked dot(s) printed on it.
The following images show the same paper being folded step by step, ending with the fully folded result where every visible point is labeled using the letters A, B, C, D.
Paper is opaque: points on the back side may be hidden by overlying layers, and the same physical dot may appear in different positions across steps as the paper folds.
If the paper rotates during the sequence, it rotates slowly.
You must track each unmarked dot through the fold sequence and report the letter(s) in the final image that correspond to the original unmarked dot(s).
If this task depends on a specific visual definition, use this definition exactly: not applicable.
The visual evidence for this question is provided below.

[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.
3. Do not output explanation beyond the required final answer.

[Question]
Which lettered point(s) in the final folded image correspond to the unmarked dot(s) shown on the flat paper in the first image? List the matching letter(s).

[Answer Format]
Output exactly one JSON object: {\"answer\": {json_answer_value}} and nothing else.
Replace {json_answer_value} with the single answer for this task, formatted according to a JSON array of uppercase letters chosen from [A, B, C, D], with no duplicates, e.g. [\"A\", \"C\"].

[Images]

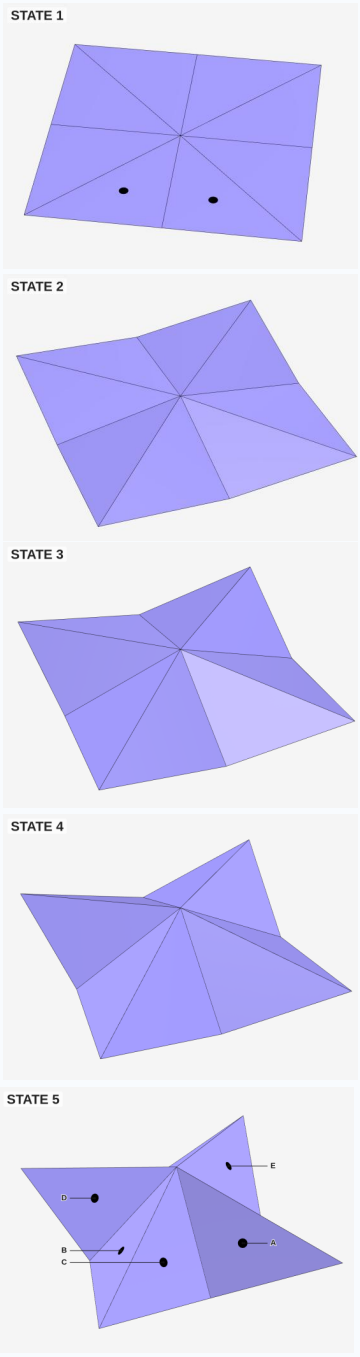


Figure 21: **Origami Point Tracking** qualitative examples across the four bases and three difficulty tiers.

Interactive Task: Continuity Pipe

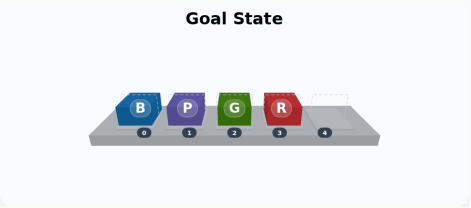
[Task]
You are solving Order Swap Puzzle.
In this task, you must swap one colored block with the empty slot per turn until the row matches the target arrangement.
In this task, you must transform the current arrangement into the goal arrangement by swapping one colored block with the blank slot at each turn.
In both images, slot indices increase from left to right from 0 through 4.

[Current Task]
The current state image is shown below.

[Image]

The goal state image is shown below.

[Image]



[Rules]
1. At each turn, choose exactly one non-empty slot index.
2. The selected block swaps positions with the blank slot.
3. A legal action must be one of the legal actions listed for the current state.
4. After each action, the environment returns the next state. Illegal actions keep the state unchanged but still count as a step.
5. The task is solved when the current arrangement exactly matches the goal arrangement.
6. The episode ends when the puzzle reaches a done state or the step budget is exhausted.
7. At each turn, output exactly one next action for the current state.

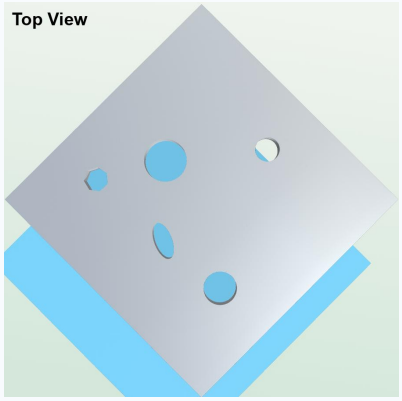
[Answer Format]
Output exactly one JSON object: {"answer": {slot_index}} and nothing else.
Replace {slot_index} with the single legal answer for this task, chosen from the legal slot indices listed for the current state, using the JSON shape {"answer": {slot_index}}.

Figure 22: **Swap Puzzle** qualitative examples.

Reasoning Task: Enclosure Hole Detection

[Task]
You are solving a topological task called Hole Detection under the enclosure category.
In this task, you must determine how many holes are visible in a top-down image of a board. Count only openings that pass through the board to open space.
A hole is an opening that connects through the board to open space. A pit or shallow depression that does not pass through the board is not a hole.
If another board is visible beneath it, evaluate only the top board and count holes on the top board only.
The visual evidence for this question is provided below.

[Image]



[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.
3. Do not output explanation beyond the required final answer.

[Question]
How many holes are visible in this top-down view of the board?

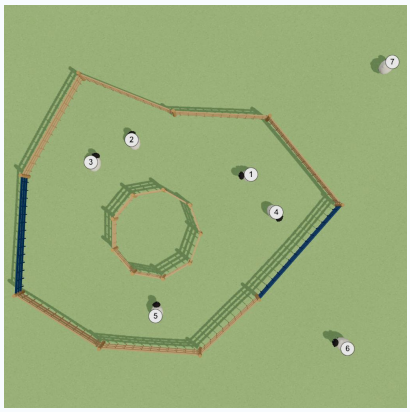
[Answer Format]
Output exactly one JSON object: {\"answer\":{ans}} and nothing else.
Replace {ans} with the single legal answer for this task, chosen from the valid integers for this task.

Figure 23: Hole qualitative examples.

Reasoning Task: Enclosure Sheep (Count Inside)

[Task]
You are solving the enclosure task.
In this task, you must look at a 3D scene with fences and numbered sheep, and count how many sheep cannot escape to the outside.
If this task depends on a specific visual definition, use this definition exactly: A sheep cannot escape if it is trapped inside fences and there is no continuous path from the sheep to the outside without crossing a fence segment. Sheep already outside all fences do not count. Sheep inside a fence with a usable gap to the outside also do not count.
The visual evidence for this question is provided below.

[Image]



[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.
3. Do not output explanation beyond the required final answer.

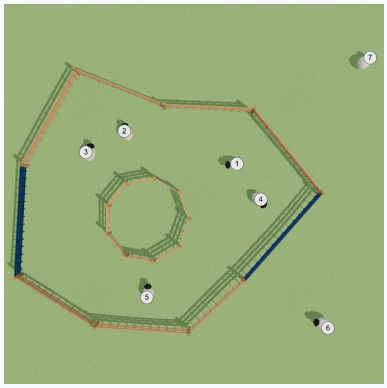
[Question]
How many sheep cannot escape to the outside? Count sheep that are trapped inside fences. Do not count sheep that are already outside all fences or sheep that can escape through a gap.

[Answer Format]
Output exactly one JSON object: {\"answer\": {value}} and nothing else.
Replace {value} with the single legal answer for this task, chosen from a single JSON integer representing the count of sheep that cannot escape to the outside, for example 3.

Reasoning Task: Enclosure Sheep (Escape Possibility)

[Task]
You are solving the enclosure task.
In this task, you must look at a 3D scene with fences and numbered sheep, and identify which sheep can escape to the outside through fence gaps.
If this task depends on a specific visual definition, use this definition exactly: An enclosure is a closed region formed by connected fence segments. A gap is a missing fence segment that creates an opening. A sheep can escape if there exists a continuous path from that sheep to the exterior that passes only through gaps and does not cross any fence segment.
The visual evidence for this question is provided below.

[Image]



[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.
3. Do not output explanation beyond the required final answer.

[Question]
Which sheep can escape to the outside through gaps in the fence? List their IDs.

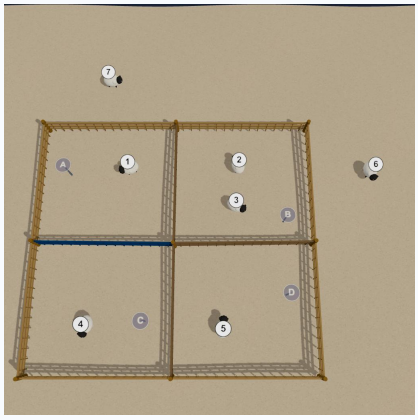
[Answer Format]
Output exactly one JSON object: {\"answer\": {value}} and nothing else.
Replace {value} with the single legal answer for this task, chosen from the JSON string \"NONE\" or a single JSON string containing the escaping sheep IDs in ascending order, separated by commas, for example \"1, 3, 5\".

Figure 24: **Enclosure-Sheep** qualitative examples.

Reasoning Task: Enclosure Sheep (Max Cell Count)

[Task]
You are solving the enclosure task.
In this task, you must look at a partitioned enclosure scene where inner fences divide the enclosure into labeled regions, and determine which region contains the most sheep.
If this task depends on a specific visual definition, use this definition exactly: A partitioned enclosure is divided into multiple labeled cells by inner fences. If multiple regions tie for the largest count, any tied label is acceptable.
The visual evidence for this question is provided below.

[Image]



[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.\n3. Do not output explanation beyond the required final answer.

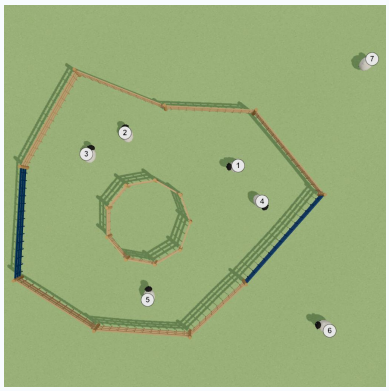
[Question]
This scene shows a partitioned fenced area with labeled regions (A, B, C, ...).
Which region contains the most sheep?

[Answer Format]
Output exactly one JSON object: {\"answer\": {value}} and nothing else.
Replace {value} with the single legal answer for this task, chosen from a single JSON string containing one region label visible in the image, such as \"A\", \"B\", or \"C\".

Reasoning Task: Enclosure Sheep (Fence Repair)

[Task]
You are solving the enclosure task.
In this task, you must look at a 3D scene with a partially broken outermost fence and determine the minimum number of fence gaps that must be repaired to fully close it.
If this task depends on a specific visual definition, use this definition exactly: An enclosure is a closed region formed by connected fence segments. A gap is a missing fence segment that breaks the outermost fence. Each distinct gap counts as one repair, and the answer is the minimum number of repairs needed so that the outermost fence becomes fully closed.
The visual evidence for this question is provided below.

[Image]



[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.
3. Do not output explanation beyond the required final answer.

[Question]
What is the minimum number of fence gaps that must be repaired to fully close the outermost fence?

[Answer Format]
Output exactly one JSON object: {\"answer\": {value}} and nothing else.
Replace {value} with the single legal answer for this task, chosen from a single JSON integer representing the minimum number of fence gaps that must be repaired, for example 2.

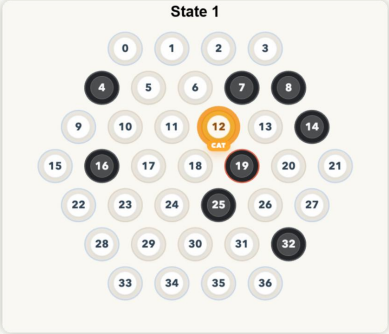
Figure 25: **Enclosure-Sheep** qualitative examples.

Interactive Task: Enclosure Chat Noir

[Task]
You are solving Chat Noir / Encircle the Cat.
In this task, you must block one hex cell per turn to surround the cat before it reaches the boundary.
In this task, you must trap the cat by blocking one open non-cat cell at each turn before the cat reaches any boundary cell.
Each cell index is printed directly on the board image.

The board radius is 3.
The current cat index is 18.
Legal actions for this turn: 0, 1, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 17, 19, 20, 21, 22, 23, 24, 26, 28, 29, 30, 31, 32, 33, 34, 35, 36.
To solve this task, output the next legal action for the current state.

[Image]



[Rules]
1. At each turn, choose exactly one open non-cat cell index to block.
2. After your action, the environment applies the block and then the cat may stay still or move one hex step according to the configured cat policy.
3. A legal action must be one of the legal actions listed for the current state.
4. Illegal actions keep the state unchanged but still count as a step.
5. The task succeeds when the cat has no remaining path to any boundary cell.
6. The task fails when the cat reaches a boundary cell.
7. At each turn, output exactly one next action for the current state.

[Answer Format]
Output exactly one JSON object: {"answer":{cell_index}} and nothing else.
Replace {cell_index} with the single legal answer for this task, chosen from the legal cell indices listed for the current state, using the JSON shape {"answer":{cell_index}}.

[Current Task]
The current state image is shown below.

Figure 26: **Chat Noir** qualitative examples.

Reasoning Task: Knots Static (Structure Classification)

[Task]
You are solving the knots task. In this task, you must look at all the ropes in the image and decide which of the five categories below the whole scene fits into. If this task depends on a specific visual definition, use this definition exactly: Two basic tests you will use:

Knot test - imagine pulling on the rope's two ends (or, if the rope has no free ends, imagine trying to flatten it onto a table).
- If the rope CAN be pulled straight / flattened into a circle with NO self-overlap and NO crossings without cutting, the rope has NO knot.
- If it stays tangled no matter how hard you pull, the rope HAS a knot.

Linked test - imagine grabbing two ropes and pulling them apart in opposite directions.
- If they catch on each other and you cannot separate them without cutting one, the two ropes are LINKED.
- If they slide off each other (even after some untangling), they are NOT linked. Being tangled or merely touching is not enough.

Now use these tests to choose ONE category:

A) Simple ring
- exactly ONE rope, AND it is a closed ring (e.g. a rubber band) with NO knot tied in it.
- If you smoothed it out it would just be a circle with no self-overlap or crossings.

B) Knot
- exactly ONE rope, AND it has at least one knot tied in it.
- The rope's two ends may be free, or the ends may be joined into a ring.
- The rope cannot be pulled straight or flattened into a circle without cutting.

C) Link
- TWO OR MORE closed-ring ropes, AND every single rope is linked to at least one other rope.
- No rope is completely free, every rope passes through some other rope and cannot be pulled away on its own.

D) Unlinked multiple ropes
- TWO OR MORE separate ropes; each individual rope may be open-ended or closed-ended, and may itself be tied into a knot.
- So on its own each rope might be a simple ring, a closed-ended knot, an open straight rope, or an open-ended knot.
- Any knots tied inside a single rope STAY tied - we are NOT asking whether each rope's own knot can be undone.
- Defining feature: NONE of the ropes is linked to any OTHER rope. You could pick up each whole rope and carry it away from all the others without cutting anything.
- Two ropes that look tangled in the picture still count as unlinked if you could separate them by moving the whole ropes around.

E) Others
- the picture combines rope structures of two or more DIFFERENT categories from A-D in the same scene.
- Different parts of the picture would each be labelled with a different category (A/B/C/D) if you described them on their own.
- Key test: if you cannot describe the WHOLE picture with a single A/B/C/D label and would need at least two different category labels to cover everything, the answer is Others.
The visual evidence for this question is provided below.

[Image]

[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.
3. Do not output explanation beyond the required final answer.

[Question]
Using the five categories above, decide which one the entire scene in this image fits into.

Answer options:
A) Simple ring
B) Knot
C) Link
D) Unlinked multiple ropes
E) Others

[Answer Format]
Output exactly one JSON object: {\"answer\": {value}} and nothing else.
Replace {value} with the single legal answer for this task, chosen from one of the JSON strings \"A\", \"B\", \"C\", \"D\", or \"E\"."

Reasoning Task: Knots Static (Component Count)

[Task]
You are solving the knots task.
In this task, you must count how many separate ropes are in the picture.
If this task depends on a specific visual definition, use this definition exactly: A 'rope' is a single continuous strand.
Important:
- If two ropes are tied together, threaded through each other, tangled up, or just lying on top of each other in the picture, they still count as TWO. We are counting physical strands, NOT how many groups would fall apart if you shook them.
- A single rope that crosses over itself (self-crossings) is still ONE, not several ones.

The visual evidence for this question is provided below.

[Image]

[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.
3. Do not output explanation beyond the required final answer.

[Question]
How many separate ropes are there in this image?

[Answer Format]
Output exactly one JSON object: {\"answer\": {value}} and nothing else.
Replace {value} with the single legal answer for this task, chosen from a single non-negative JSON integer giving the number of ropes, for example 3."

Figure 27: **Knots** qualitative examples.

Reasoning Task: Knots Static (Link Topology)

[Task]
You are solving the knots task.
In this task, you must look at how the rings in the image are caught onto each other and pick which pattern matches the whole picture.
If this task depends on a specific visual definition, use this definition exactly: All ropes here are closed rings, like rings on a key chain. We say two rings are 'linked' when they go through each other so that you cannot pull them apart no matter how much you wiggle them — the only way to separate them would be to cut one open. Rings that just touch, sit next to each other, or look tangled but would slide apart with enough wiggling are NOT linked.
Decide which of the four patterns below the WHOLE picture matches:
- 'Not linked' = no two rings go through each other; every ring could be lifted off the others without cutting (even if they look tangled, if they would slide apart, they still count as not linked).
- 'Paired' = exactly TWO rings are caught through each other (like two key rings hooked together). If there are any extra rings in the picture, those extras are NOT caught on anything.
- 'Chain' = THREE OR MORE rings hooked together one after another, like a metal chain (the kind made of interlocking metal rings, e.g. a bicycle chain or a dog-leash chain). Each ring is only caught on its direct neighbour(s) in the line — not on rings further down the chain.
- 'All interlocked (Borromean-style)' = THREE rings woven together as a group. If you looked at any TWO of them by themselves they would NOT be caught on each other, but the three together still cannot be pulled apart unless one is cut. (Imagine three rings braided together so that each one holds the other two in place.)
The visual evidence for this question is provided below.

[Image]

[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.
3. Do not output explanation beyond the required final answer.

[Question]
Which of the four patterns above best describes how the rings in this image are caught onto each other?

Answer options:
A) Not linked — no two rings go through each other; every ring could be lifted off the rest without cutting
B) Paired — exactly two rings are hooked together; any extra rings are loose and not caught on anything
C) Chain — three or more rings hooked one after another in a line (like a metal chain made of interlocking rings, e.g. a bicycle chain), where each ring is caught only on its direct neighbour(s)
D) All interlocked — three rings braided together so the three as a group cannot be pulled apart, even though any two of them on their own would NOT be caught on each other

[Answer Format]
Output exactly one JSON object: {\"answer\": {value}} and nothing else. Replace {value} with the single legal answer for this task, chosen from one of the JSON strings \"A\", \"B\", \"C\", or \"D\"."

Figure 28: **Knots** qualitative examples.

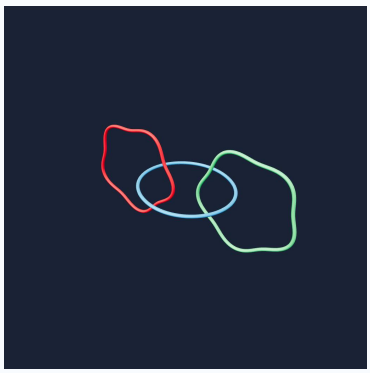
Reasoning Task: Knots Static (Link Property)

[Task]
You are solving the knots task.
In this task, you must imagine cutting and removing the ring of a specified color, then list the colors of every remaining ring that is now FREE (not linked to any other remaining ring).
If this task depends on a specific visual definition, use this definition exactly:
Setup:
- Each ring in this picture has a distinct solid color, drawn from the palette: red, blue, green, brown, white, purple.
- You will be told the color of one specific ring. Imagine you cut that ring open and pull it out of the picture.
- Look at all the rings that REMAIN.

Definitions:
- 'Linked' = two rings pass through each other and cannot be pulled apart no matter how much you wiggle them; the only way to separate them would be to cut one open. Rings that just touch, sit next to each other, or look tangled but would slide apart with enough wiggling are NOT linked.
- A remaining ring is FREE if it is not linked to ANY other remaining ring; you could pick it up and carry it away from all the other remaining rings without cutting anything.
- A remaining ring is NOT free if it is still linked to at least one other remaining ring.

What to output:
- List the colors of every remaining ring that is now FREE.
- Use only the color names from the palette: red, blue, green, brown, white, purple.
- The colors may be listed in any order.
- Do NOT include the color of the ring that was removed.
- If a remaining ring is still linked to at least one other remaining ring, do NOT include its color.
- If NO remaining ring is free (every remaining ring is still linked to at least one other), output exactly [\"none\"]. Do NOT output an empty list.
The visual evidence for this question is provided below.

[Image]



[Rules]
1. Use only the images and text provided in this prompt.
2. If answer options are provided, choose only from the provided options.
3. Do not output explanation beyond the required final answer.

[Question]
Imagine you cut and remove the red ring from this configuration. After removing it, which of the remaining rings are now FREE (not linked to any other remaining ring)? List the colors of all rings that are now free.

[Answer Format]
Output exactly one JSON object: {\"answer\": {value}} and nothing else. Replace {value} with the single legal answer for this task, chosen from a JSON list of one or more strings drawn from {\"red\", \"blue\", \"green\", \"brown\", \"white\", \"purple\", \"none\"}; list every remaining ring that is now free; colors may appear in any order; for example [\"red\", \"blue\"]; if NO remaining ring is free output exactly [\"none\"] (never an empty list)."

Figure 29: **Knots** qualitative examples.

Interactive Task: Knots Untangle

[Task]
You are solving Knots Untangle. In this task, you must untangle the ropes by moving one endpoint at a time so no two ropes cross.
The pegboard is a square grid of indexed holes. Each rope connects two holes. A move relocates one endpoint from its current hole to an empty hole. Two ropes count as crossing when their top-down projected paths overlap, including cases where rope bodies touch because of rope thickness.

Current state summary:
- State index: 0
- Difficulty: easy
- Pegboard: 5x5 grid (rows/columns 0..4)
- Coordinate convention: holes are addressed as (row, col). Row numbers are shown along the top edge and increase from left to right; column numbers are shown along the left edge and increase from top to bottom. The top-left hole is (row=0, col=0).
- Rope count: 4
- Crossings remaining: 1
- Goal: reduce crossings to 0

Ropes on the board:
- Rope 0 (red) endpoints at (row=0,col=1) and (row=4,col=1)
- Rope 1 (green) endpoints at (row=0,col=4) and (row=4,col=4)
- Rope 2 (blue) endpoints at (row=2,col=0) and (row=4,col=2)
- Rope 3 (yellow) endpoints at (row=0,col=3) and (row=4,col=3)

Occupied holes:
(row=0,col=1), (row=4,col=1), (row=0,col=4), (row=4,col=4), (row=2,col=0), (row=4,col=2), (row=0,col=3), (row=4,col=3)

Legal actions for this turn: not provided; infer one legal move from the current state image and task rules
Output ONLY a single JSON object in this exact format and nothing else: {"answer":{"src_row":R,"src_col":C,"tgt_row":R,"tgt_col":C}}

[Rules]
1. At each turn, infer one legal move from the current image and task information. No legal-action list will be provided.
2. A legal action must move one rope endpoint from an occupied source hole to an empty target hole.
3. After each action, the environment returns the next state. Illegal actions keep the board state unchanged but still count as a step.
4. The task is solved when zero crossings remain.
5. The episode ends when the puzzle is solved or when the step budget (15) is exhausted.
6. At each turn, output exactly one next action for the current state.

[Answer Format]
Output exactly one JSON object: {"answer": {value}} and nothing else. Replace {value} with the single legal answer for this task, chosen from a JSON object with integer fields "src_row", "src_col", "tgt_row", and "tgt_col" describing one legal move for the current turn, for example {"src_row":0,"src_col":1,"tgt_row":2,"tgt_col":2}.

[Current Task]
The current state image is shown below.

[Image]

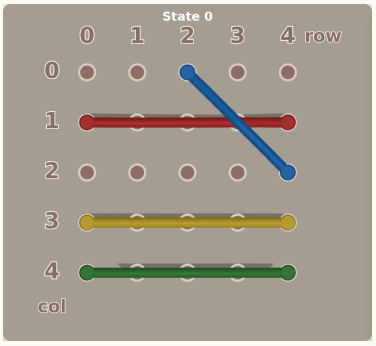


Figure 30: **Knots Untangle** qualitative examples.

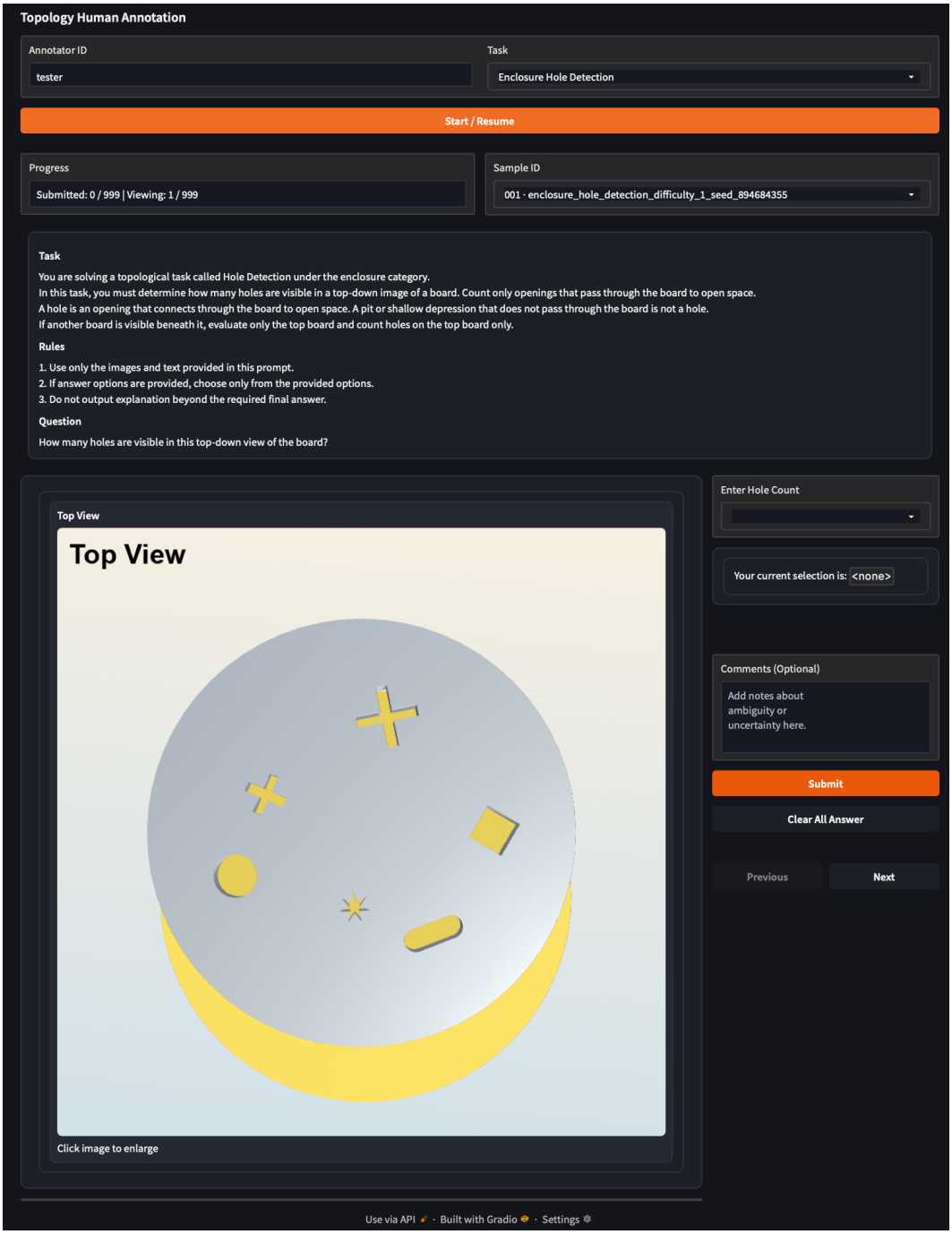


Figure 31: Human annotation interface for reasoning tasks. Annotators are shown the same visual inputs and task instructions used in model evaluation, together with a task-specific answer widget and an optional comment field for ambiguous cases.

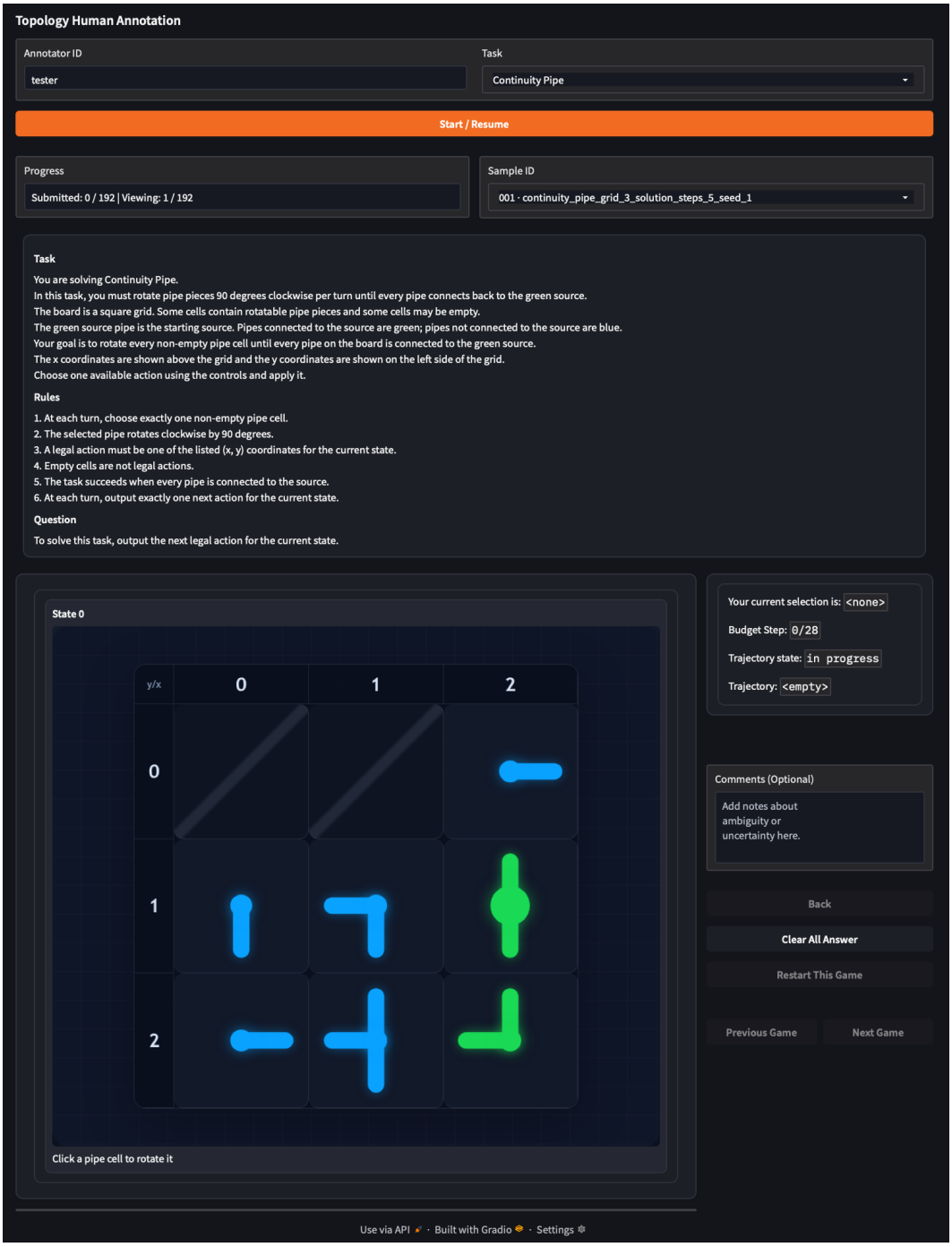


Figure 32: Human annotation interface for planning tasks. Annotators interact with the same browser-based environment used by the model runner. Each episode is initialized from the JSONL reset configuration, actions are recorded by the interface, and success is determined from the environment terminal state.

[Task] You are solving a continuity task. Determine which locations lie in the same connected component, or which single manipulation reconnects two disconnected locations.
[Rules] Use only the provided image(s). Do not cross walls, closed doors, blocking bars, or obstacle boundaries.
[Question] Return the requested reachable labels, removable bars, or connection decision.
[Answer Format] Output exactly one JSON object: `{"answer": value}`.

Figure 33: Prompt for *Continuity* reasoning and planning tasks.

[Task] You are solving a separation task. Identify whether visible parts belong to one connected object/subassembly, or choose a legal next action that keeps a single continuous stroke valid.
[Rules] Treat contact, overlap, and connected geometry according to the task definition; do not infer hidden connectors.
[Question] Select the matching option or next move.
[Answer Format] Output exactly one JSON object: `{"answer": value}`.

Figure 34: Prompt for *Separation* tasks.

[Task] You are solving an order task. Read the sequence along a continuous carrier, track point labels through a fold sequence, or plan swaps that transform the current order into the goal order.
[Rules] Order is defined along the string, paper surface, or puzzle track rather than by raw image coordinates.
[Question] Return the requested sequence, relation, visible point set, or next action.
[Answer Format] Output exactly one JSON object: `{"answer": value}`.

Figure 35: Prompt for *Order* tasks.

[Task] You are solving an enclosure task. Decide which objects are inside a closed boundary, whether an object can escape through gaps, how many through-holes a solid has, or where to place a blocking action.
[Rules] A region is enclosed only when every continuous path to the outside crosses a boundary; a hole must pass through the object.
[Question] Return the requested count, label set, repair choice, or action.
[Answer Format] Output exactly one JSON object: `{"answer": value}`.

Figure 36: Prompt for *Enclosure* tasks.

[Task] You are solving a knots task. Determine whether the visible strand configuration is knotted, linked, unlinked, or composed of multiple components, or choose a legal endpoint move that untangles it.
[Rules] Do not allow strands to pass through each other; crossing over/under evidence determines the topology.
[Question] Return the requested class, component count, link property, or next action.
[Answer Format] Output exactly one JSON object: `{"answer": value}`.

Figure 37: Prompt for *Knots* tasks.

Camera factor	Diagnostic role
Top vs. oblique	Tests whether models preserve connectivity, enclosure, and knot over/under relations when Euclidean projection changes.
Field of view	Separates global topology errors from missed off-screen or cropped boundary segments.
Zoom/crop	Identifies failures caused by small gaps, thin walls, or subtle through-holes rather than by task semantics.
Multi-view pair	Checks whether an answer is stable when one view exposes depth or occlusion cues missing from another view.

Figure 38: Sensitivity of topological reasoning to camera configurations. The same seed can be rendered under controlled view changes while keeping the topological ground truth fixed.

Bias	Where it appears	Observable symptom
Connectivity by default	2D Maze, 3D Maze, Pipe, Chat-Noir	Treats nearby regions as reachable or selects an illegal transition through a barrier.
Hole undercounting	Hole Detection, Sheep escape/repair	Counts visible depressions as filled, or misses occluded through-openings.
Crossing/chirality collapse	Knots, Untangle	Ignores over/under evidence or proposes a move that changes knot/link topology.
Coordinate-order shortcut	Bead, Origami Point, Swap	Reads image left-to-right order instead of order along the carrier or transformed surface.

Figure 39: Evidence for systematic topological biases in foundation models. The patterns are qualitative diagnostics supported by the error categories in Appendix C.

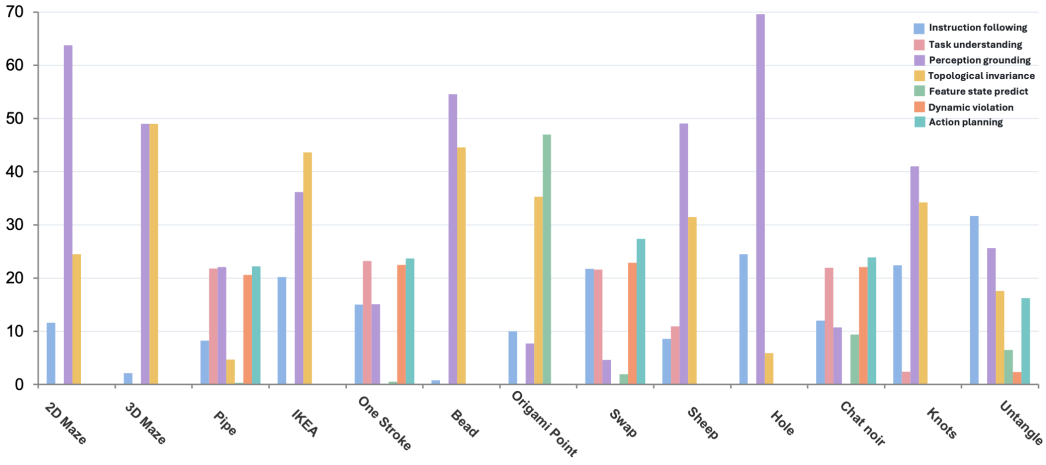


Figure 40: **Environment-level error distributions.** For each environment, errors from Gemini-3.1-Pro and InternVL3.5-241B-A28B are pooled using the number of errors as weights, and bars show the percentage assigned to each error category. The breakdown reveals that dominant failure modes vary substantially across topological environments.

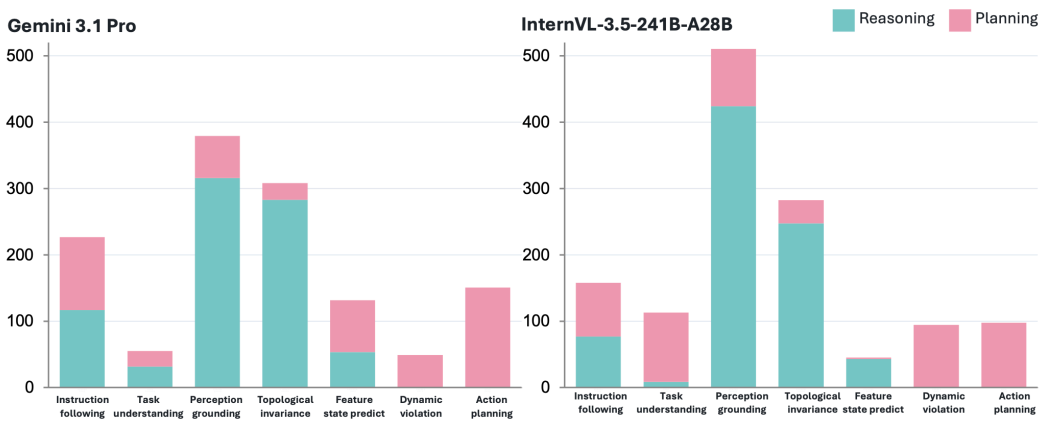


Figure 41: **Model-level error distributions across reasoning and planning.** For each model and error category, bar height sums the per-environment category percentages across reasoning and planning environments. Colors indicate whether the contribution comes from reasoning or planning tasks, exposing different failure profiles between the proprietary and open-weight models.